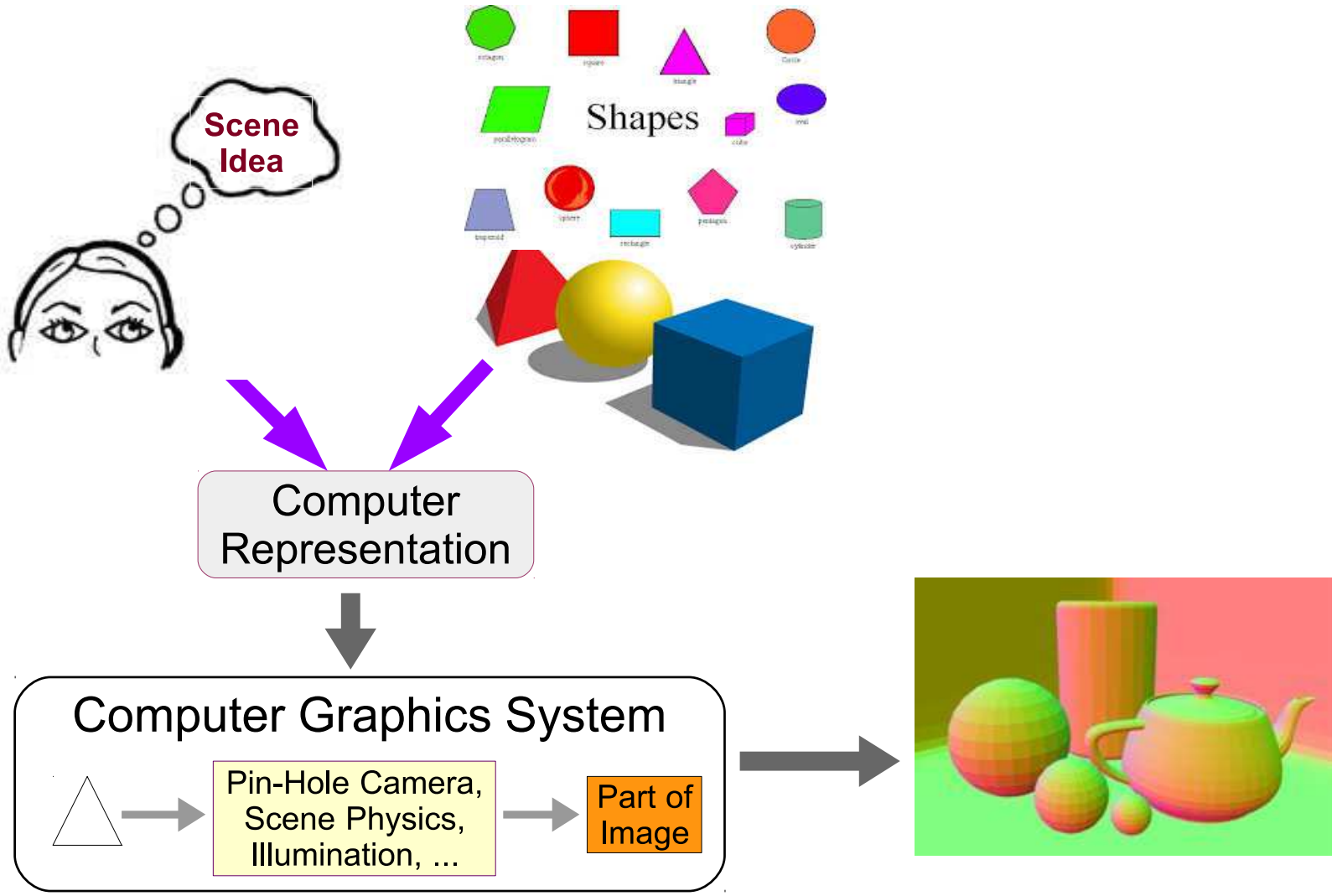
CS7.302: Computer Graphics

Geometry Module

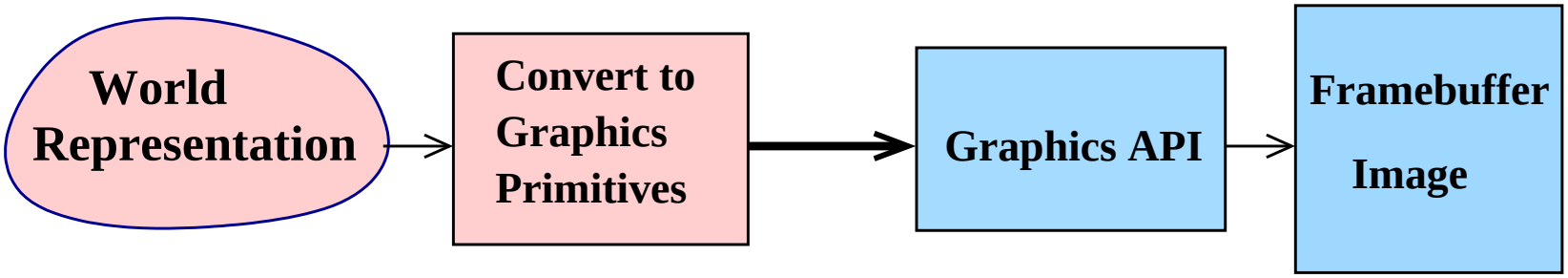
G S Ragavendra & P. J. Narayanan

Spring 2026

Graphics Process



Graphics Process

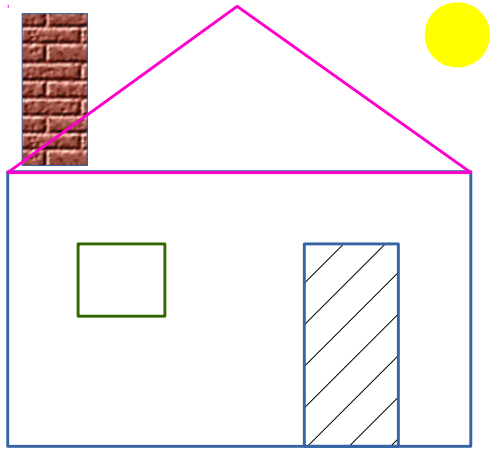


- **Model** the desired world in your head.
- **Represent** it using natural structures in the program. Convert to standard primitives supported by the API
- **Processing** is done by the API. Converts the primitives in stages and forms an image in the framebuffer
- The image is displayed automatically on the device

Drawing A House

- Compose using basic shapes

```
// Main part
drawRectangle(v1, v2, v3, v4);
drawTriangle(v2, v3, v5);      // Roof
drawRectangle( ... );          // Door
drawRectangle( ... );      // Window
drawRectangle( ... );     // Chimney
drawCircle( ... );                // Sun
```



- Convert them to OpenGL primitives

Graphics Primitives

- Graphics is concerned with the **appearance** of the 3D world to a camera
- Only *outer surface* of objects important, not interiors!!
- Hence, uses only 1D and 2D primitives

- Points: 2D or 3D. (x, y) or (x, y, z).
- Lines: specified using end-points
- Triangles/Polygons: specified using vertices
- Why not **circles, ellipses, hyperbolas**?

Graphics Attribute

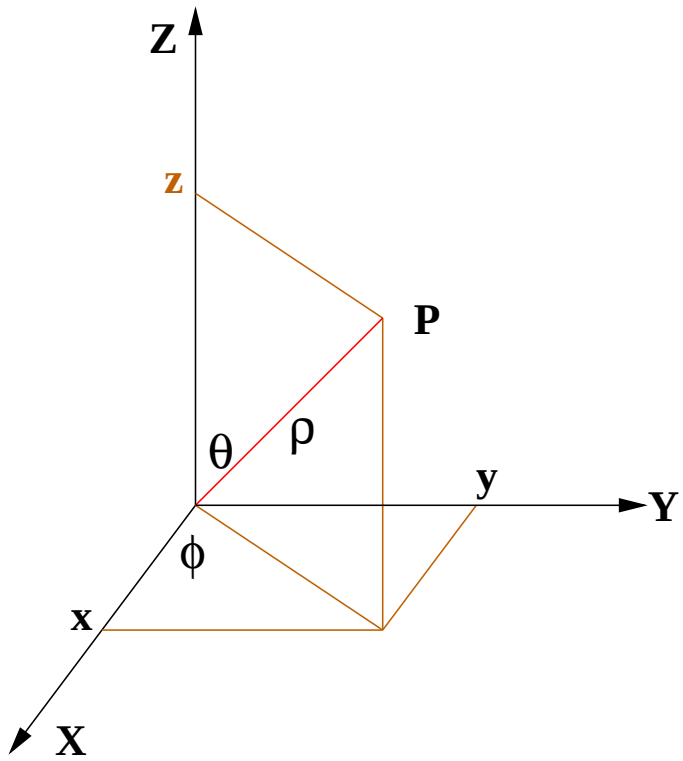
- Colour, Point width
- Line width, Line style, Line Colour
- Fill, Fill Pattern.
- Line: Give two endpoints
- Triangle: Give three vertices
- **Point** is the basic primitive that defines shape. Everything else is built using points!

Point Representation

- Use 2 or 3 numbers $(x, y, [z])$ that are the projections on to the respective coordinate axes.
- Can also be reprsented as a 2 or 3 vector $\mathbf{P}$.
- Represented using byte, short, int, float, double, etc.
- The scale and unit are application dependent.
  Could be **angstroms** or **lightyears**!
- Points undergo transformations:
  Translations, Rotations, Scaling, Shearing.

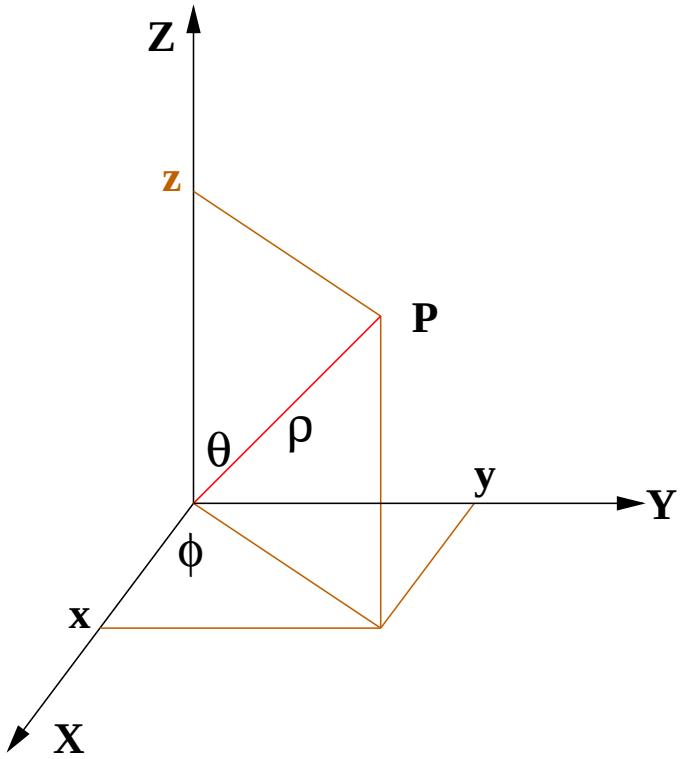
3D Coordinates

- Vector $\mathbf{P}$
- Cartesian: (x, y, z)
- Polar: (ρ, θ, ϕ)
- $z =$
  $y =$
  $x =$
- $\rho =$
  $\phi =$
  $\theta =$
- Elevation: θ, Azimuthal: ϕ



3D Coordinates

- Vector $\mathbf{P}$
- Cartesian: (x, y, z)
- Polar: (ρ, θ, ϕ)
- $z = \rho \cos\theta,$
  $y = \rho \sin\theta \sin\phi$
  $x = \rho \sin\theta \cos\phi$
- $\rho^2 = x^2 + y^2 + z^2,$
  $\phi = \tan^{-1}(y/x),$
  $\theta = \tan^{-1}(\sqrt{x^2 + y^2}/z)$
- Elevation: θ, Azimuthal: ϕ

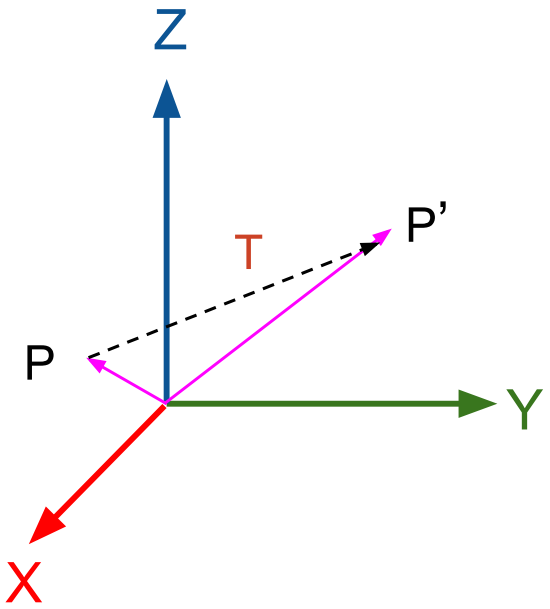


Translation

- Translate a point $P = (x, y, [z])$ by $(a, b, [c])$.
- Points coordinates become $P' = (?, ?, ?)$.
- In vector form, $P' = ?$

Translation

- Translate a point $P = (x, y, [z])$ by $(a, b, [c])$.
- Points coordinates become $P' = (?, ?, ?)$.
- In vector form, $P' = ?$



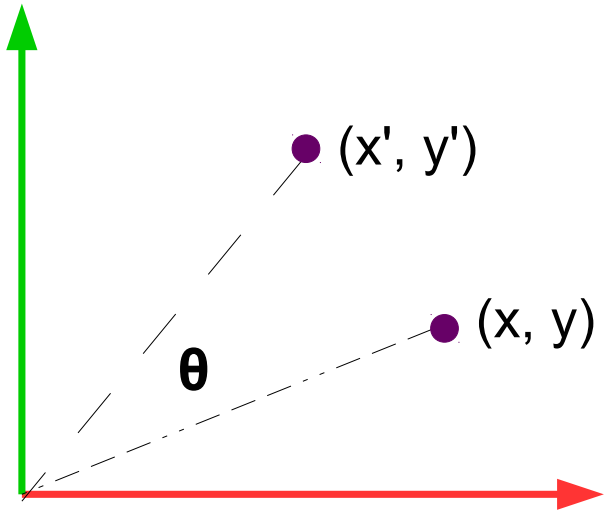
Translation

- Translate a point $P = (x, y, [z])$ by $(a, b, [c])$
- Points coordinates become $P' = (x + a, y + b, [z + c])$
- In vector form, $P' = P + T$, where $T = (a, b, [c])$
- What happens to an object or a shape made up of multiple points when all of them translate together?
- Distances, angles, parallelism are all maintained. They are *invariants*!

2D Rotation

- Rotate about origin CCW by θ
- $x' = ?, \quad y' = ?$
- Matrix notation: $P' = R\ P$

$$\begin{bmatrix} x \\ y \end{bmatrix}' = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

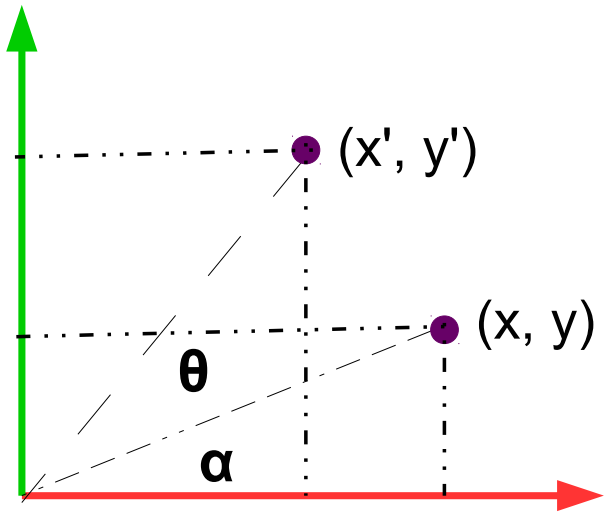


2D Rotation

- Rotate about origin CCW by θ.
- $x' = ?, \quad y' = ?$
- Matrix notation: $P' = R\ P$

$$\begin{bmatrix} x \\ y \end{bmatrix}' = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

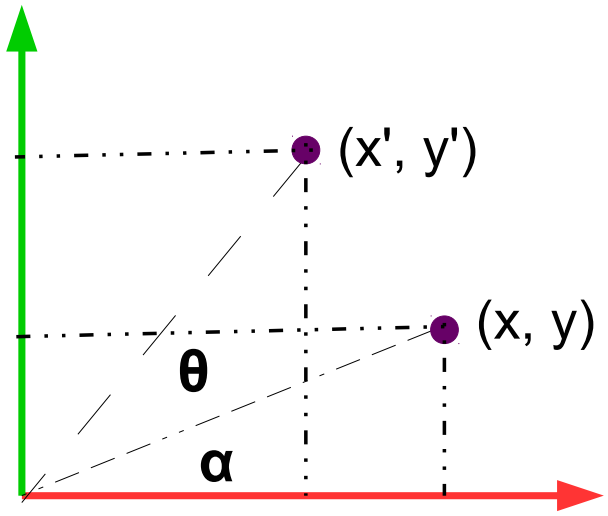
- $x = \rho \cos \alpha. \quad y = \rho \sin \alpha$
- $x' = ? \cos ?. \quad y' = ? \sin ?$



2D Rotation

- Rotate about origin CCW by θ.
- $x' = x\cos\theta - y\sin\theta$,
  $y' = x\sin\theta + y\cos\theta$.
- Matrix notation: $P' = R\,P$

$$\begin{bmatrix} x \\ y \end{bmatrix}' = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

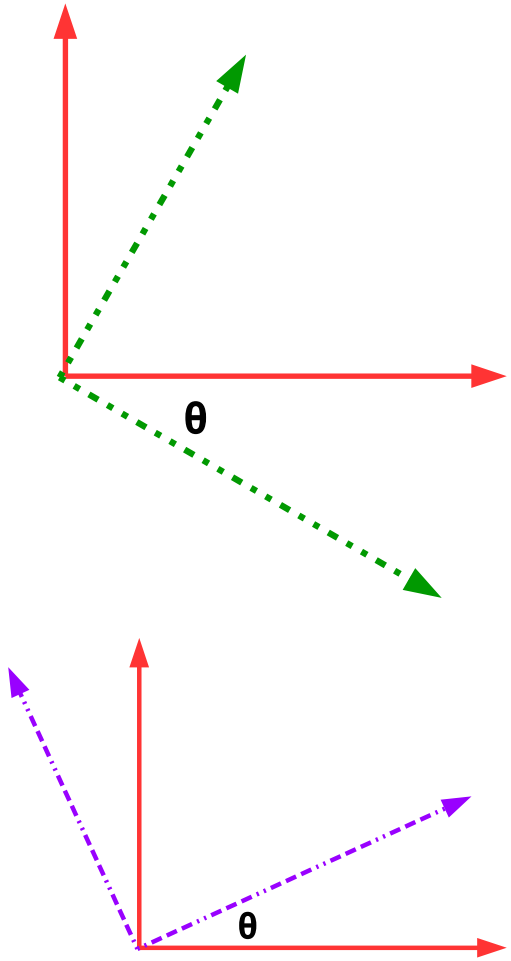


- Invariants: distances, angles, parallelism.

2D Rotation: Observations

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

- Orthonormal: $R^{-1} = R^T$
- Rows: vectors that **rotate to** coordinate axes
- Cols: vectors coordinate axes **rotate to**



3D Rotations

- Rotation could be about any axis in 3D! What does it mean?
  - Distance of each point to the rotation axis remains same
  - Each point moves in a circle on a plane perpendicular to the rotation axis, with its centre on the axis
- About Z-axis: Just like 2D rotation case. Z-coordinates don't change anyway.
- X-Y coordinates change exactly the same way as in 2D.
- CCW +ve, looking into the **arrowhead:** $R_z(\theta) = ??$

3D Rotations

- About Z-axis: Z-coordinates don't change anyway

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- CCW +ve; orthonormal; length preserving
- Rows: vectors that rotate onto axes; columns: vectors that axes rotate into....
- Rotation could be about any axis in 3D!

3D Rotations

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \qquad R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

- CCW +ve; orthonormal
- Rows: vectors that rotate onto axes; columns: vectors that axes rotate into....
- Rotation about an arbitrary axis, for example, $[1, 1, 1]^\intercal$?? **Coming soon ....**

Non-uniform Scaling

- Scale along X, Y, Z directions by s, u, and t:
  $x' = s\,x, \;\; y' = u\,y, \;\; z' = t\,z$.
- We are more comfortable with $P' = S\;P$ or

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}' = \begin{bmatrix} s & 0 & 0 \\ 0 & u & 0 \\ 0 & 0 & t \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- Rectangles can become squares and vice versa!
- Invariants: parallelism, ratios of lengths in any direction (Angles also for uniform scaling.)

Shearing

- Add a little bit of x to y or other combinations

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}' = \begin{bmatrix} 1 & x_y & x_z \\ y_x & 1 & y_z \\ z_x & z_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- One of $x_y, x_z, y_x, y_z, z_x, z_y \neq 0$.
- Rectangles can become parallelograms, but not general quadrilaterals
- Invariants: parallelism, ratios of lengths in any direction.

Reflection

- Negative entries in a matrix indicate reflection.

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}' = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- Reflection needn't be about a coordinate axis alone

Summary of Transformations

- Translation: New coordinates $\mathbf{P'} = \mathbf{P} + \mathbf{t}$
- Rotation: $\mathbf{P'} = \mathbf{R}\,\mathbf{P}$
- Scaling: $\mathbf{P'} = \mathbf{S}\,\mathbf{P}$
- Shearing: $\mathbf{P'} = \mathbf{S_h}\,\mathbf{P}$
- Reflection: $\mathbf{P'} = \mathbf{R_f}\,\mathbf{P}$
- Each is a matrix-vector product, except ....

General Transformation

- Rotation, scaling, shearing, and reflection: **Matrix-vector** product. Vectors get tranformed into other vectors
- Translation alone is a **vector-vector** addition
- Sequence of: Translation, rotation, scaling, translation and rotation: $\mathbf{P'} = \mathbf{R_2}\left[\mathbf{S}\ \mathbf{R_1}\ (\mathbf{P} + \mathbf{t_1}) + \mathbf{t_2}\right]$
- If translation is also a matrix-vector product, we can combine above transformations into a single matrix: $\mathbf{P'} = \mathbf{R_2}\ \mathbf{T_2}\ \mathbf{S}\ \mathbf{R_1}\ \mathbf{T_1}\ \mathbf{P} = \mathbf{M}\ \mathbf{P}$
- How? Answer: **homogeneous coordinates**!

Homogeneous Coordinates

- Add a *non-zero scale factor* w to each coordinate.
  A 2D point is represented by a vector $[x \;\; y \;\; w]^\intercal$
- $[x \;\; y \;\; w]^\intercal \equiv (x/w, \; y/w)$.
- Simplest value of w is obviously 1
- Translate $[x \;\; y]^\intercal$ by $[a \;\; b]^\intercal$ to get $[x+a \;\; y+b]^\intercal$

$$\begin{bmatrix} x+a \\ y+b \\ 1 \end{bmatrix} = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translation as a Matrix

- Add a *non-zero scale factor* w to each coordinate.
  A 2D point is represented by a vector $[x \;\; y \;\; w]^\intercal$
- Translate $[x \;\; y]^\intercal$ by $[a \;\; b]^\intercal$ to get $[x+a \;\; y+b]^\intercal$

$$\begin{bmatrix} x+a \\ y+b \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Now, translation is also: $\mathbf{P'} = \mathbf{T}\ \mathbf{P}$, a matrix-vector product and a linear operation.

Sequence of Transformations

- Now, translation is also: $\mathbf{P' = T\ P}$
- Sequence of: Translation, rotation, scaling, translation and rotation: $\mathbf{P' = R_2\ T_2\ S\ R_1\ T_1\ P = M\ P}$
- Arbitrary long sequence of transformations reduce to a single matrix M!
- Similarly for 3D. Points represented by: $[x\ y\ z\ w]^T$.
- All matrices are 3×3 in 2D. Last row is $[0\ \ 0\ \ 1]$.
- All matrices are 4×4 in 3D. Last row is $[0\ \ 0\ \ 0\ \ 1]$.

Homogeneous Representation in 3D

- Convert to a 4-vector with a scale factor as fourth. $(x, y, z) \equiv [kx \;\; ky \;\; kz \;\; k]^\intercal$ for any $k \neq 0$.
- Translation, rotation, scaling, shearing, etc. become linear operations represented by 4×4 matrices.
- What does $[x \; y \; z \; 0]^\intercal$ mean?
- $[a \; b \; c \; d]^\intercal$ could be a point or a plane.
  Lines are specified using two vectors: either as the join of two points or as the intersection of two planes!

Transformation Matrices: Rotations

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- CCW +ve; orthonormal; length preserving; rows give direction vectors that rotate onto each axis; columns ....

Translation, Scaling, Composite

$$T(a,b,c) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(a,b,c) = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- A sequence of transforms can be represented using a composite matrix: $\mathbf{M = R_x T R_y S T \cdots}$
- Operations are not commutative, but are associative.
- $\mathbf{RT}$ and $\mathbf{TR}$??

Rotation and Translation

- $T_{4\times4} = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$ and $R_{4\times4} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix}$
- $T\,R = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} = ?$
- $R\,T = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = ?$

Rotation and Translation

- $T_{4\times4} = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$ and $R_{4\times4} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix}$
- $T\,R = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$
- $R\,T = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{Rt} \\ \mathbf{0} & 1 \end{bmatrix}$
- $TR = R\,T$ if: (a) $\mathbf{R} = \mathbf{I}$ or (b) $\mathbf{t} = \mathbf{0}$ or (c) $\mathbf{Rt} = ?$

Rotation and Translation

- $T_{4\times4} = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$ and $R_{4\times4} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix}$
- $T\ R = \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$
- $R\ T = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{Rt} \\ \mathbf{0} & 1 \end{bmatrix}$
- $TR = R\,T$ if: (a) $\mathbf{R} = \mathbf{I}$ or (b) $\mathbf{t} = \mathbf{0}$ or (c) $\mathbf{Rt} = \mathbf{t}$
- When is $\mathbf{Rt} = \mathbf{t}$? $\mathbf{t}$ is an eigenvector of $\mathbf{R}$
- **Question:** Are transformations *commutative*?

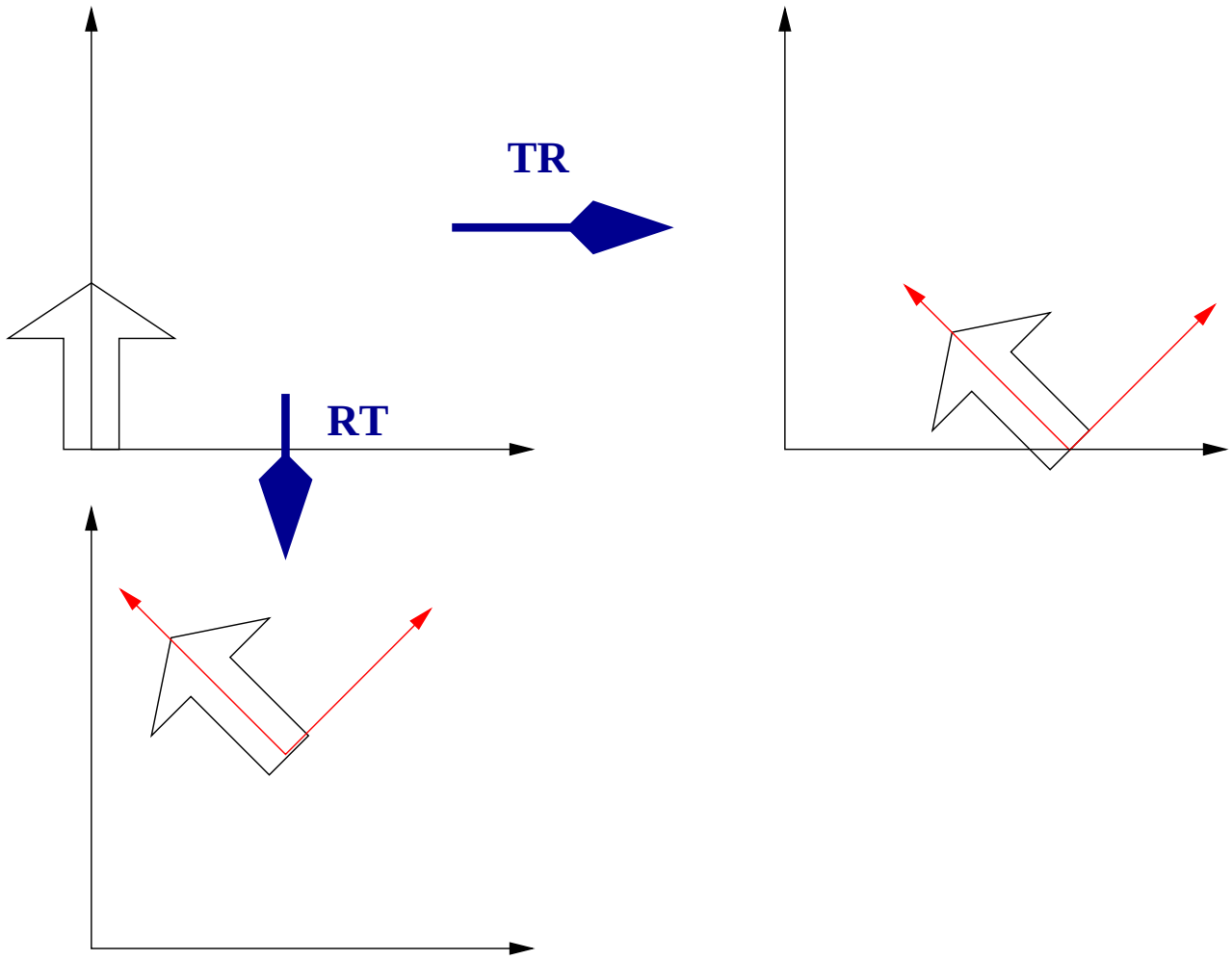
Commutativity

- Translations are commutative: $\mathbf{T_1 T_2 = T_2 T_1}$
- Scaling is commutative: $\mathbf{S_1 S_2 = S_2 S_1}$
- Are rotations commutative? $\mathbf{R_1 R_2} \stackrel{?}{=} \mathbf{R_2 R_1}$
- Rotation and Scaling commute? $\mathbf{SR} \stackrel{?}{=} \mathbf{RS}$
- What would be an example?

Commutativity

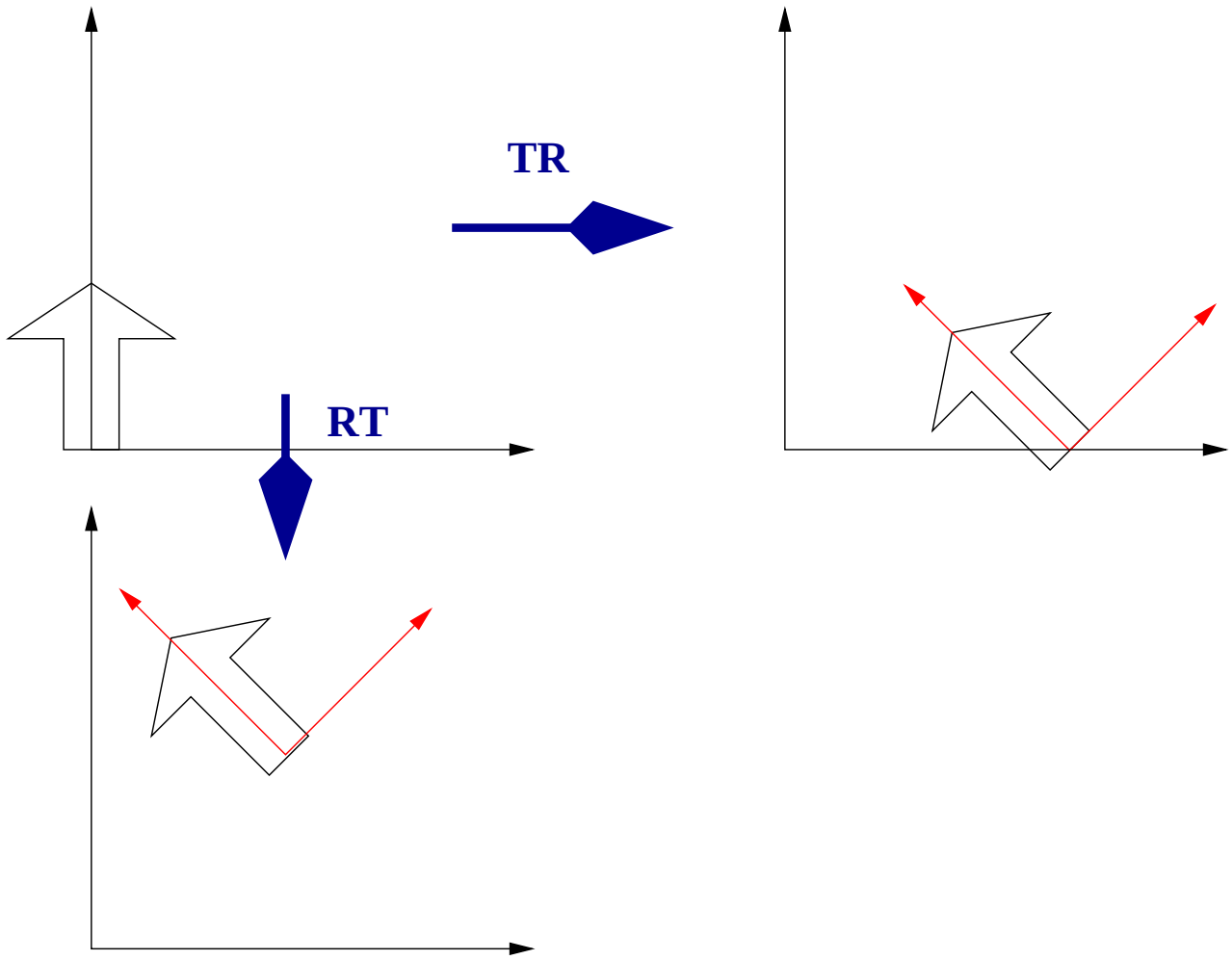
- Translations are commutative: $\mathbf{T_1 T_2 = T_2 T_1}$
- Scaling is commutative: $\mathbf{S_1 S_2 = S_2 S_1}$
- Are rotations commutative? $\mathbf{R_1 R_2 \neq R_2 R_1}$
- Rotation and Scaling commute. $\mathbf{SR = RS}$
- Consider the effect on Z-axis of
  $\mathbf{R_x(90) R_y(90)}$ and $\mathbf{R_y(90) R_x(90)}$
- $\mathbf{RT \neq TR}$. (If translation is not parallel to rotation axis)
- Consider: $\mathbf{R}(\pi/4)$ and $T(5, 0)$.
  Where does the origin $(0, 0)$ go in $\mathbf{TR}$ and $\mathbf{RT}$?

TR and RT



TR **RT**

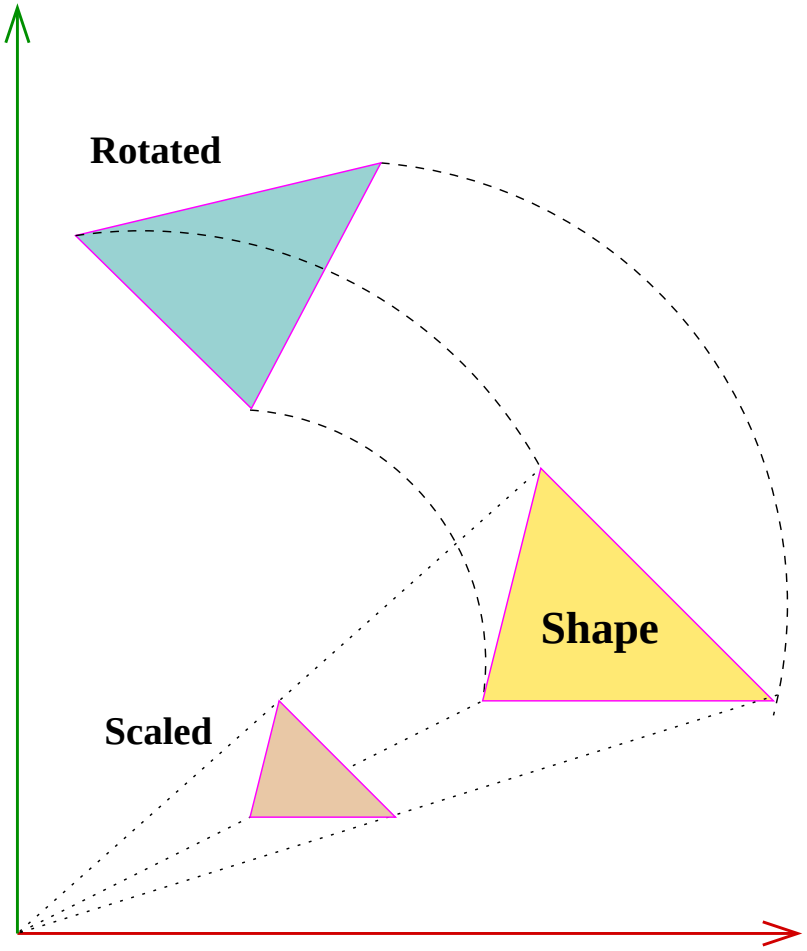
TR and RT



$\mathbf{TR}$ keeps it on X axis to $(5, 0)$. $\mathbf{RT}$ takes it to $(\frac{5}{\sqrt{2}}, \frac{5}{\sqrt{2}})$.

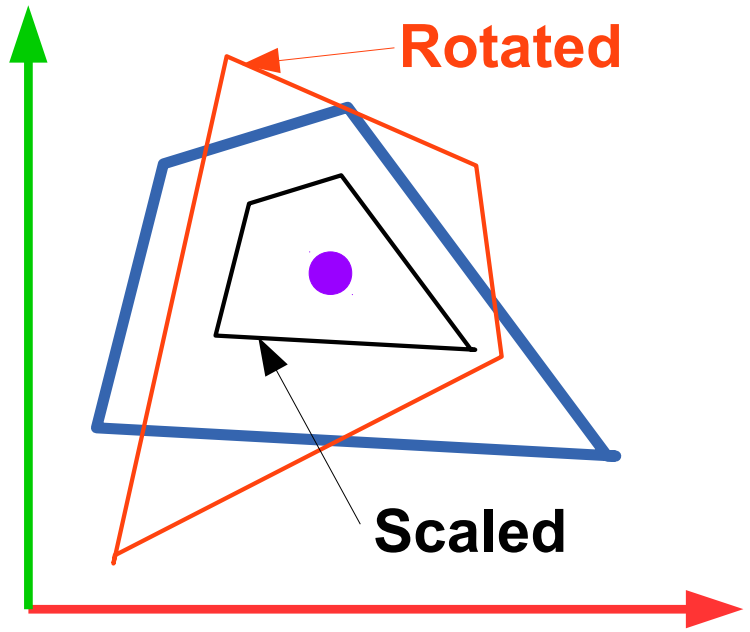
Objects Away from Origin

- Object "translates" when rotated or scaled!!
- Default: Perform these **about the origin**
- How do we transform points "in place"?
- Rotate or scale about the centroid of the object. Or about an arbitrary point
- How?



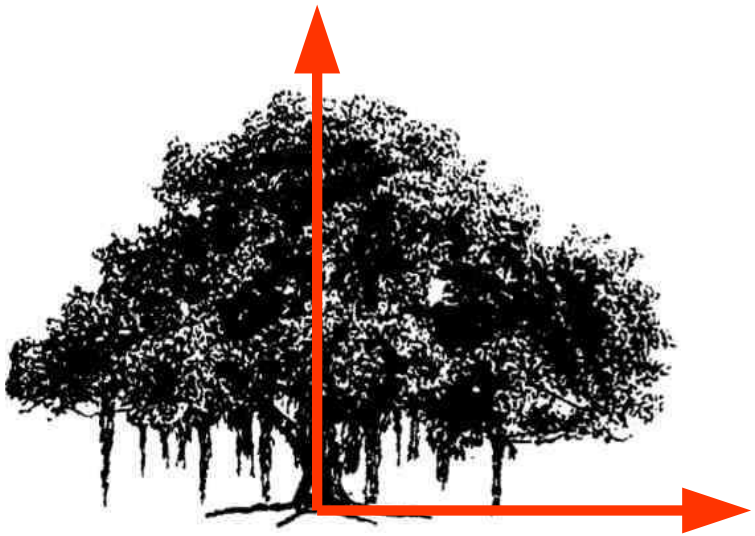
Transformations About A Point

- Rotating about point P
  - Align P with origin
  - Rotate/scale about origin
  - Translate back
- Rotation:
  $\mathbf{R_C}(\theta) = \mathbf{T(C)}\ \mathbf{R}\ \mathbf{T(-C)}$
- Scaling:
  $\mathbf{S_C}() = \mathbf{T(C)}\ \mathbf{S}()\ \mathbf{T(-C)}$
- A transformation $\mathbf{M}$:
  $\mathbf{M_C} = \mathbf{T(C)}\ \mathbf{M}\ \mathbf{T(-C)}$

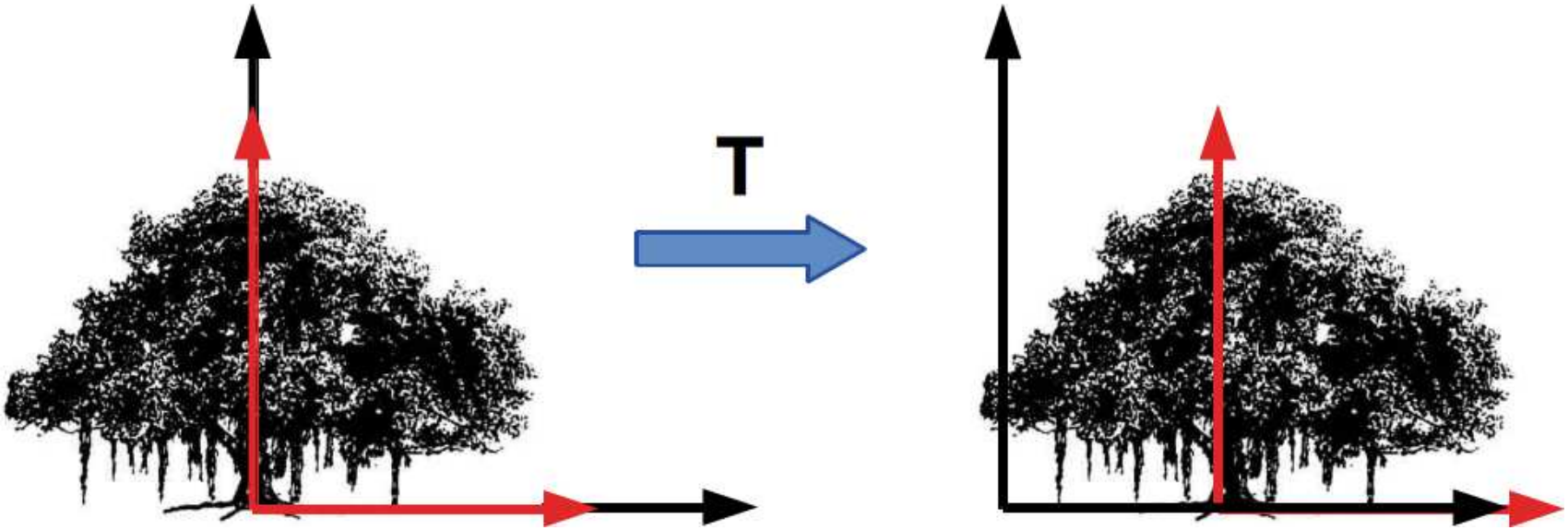


Object

- Object has a coordinate frame of its own.



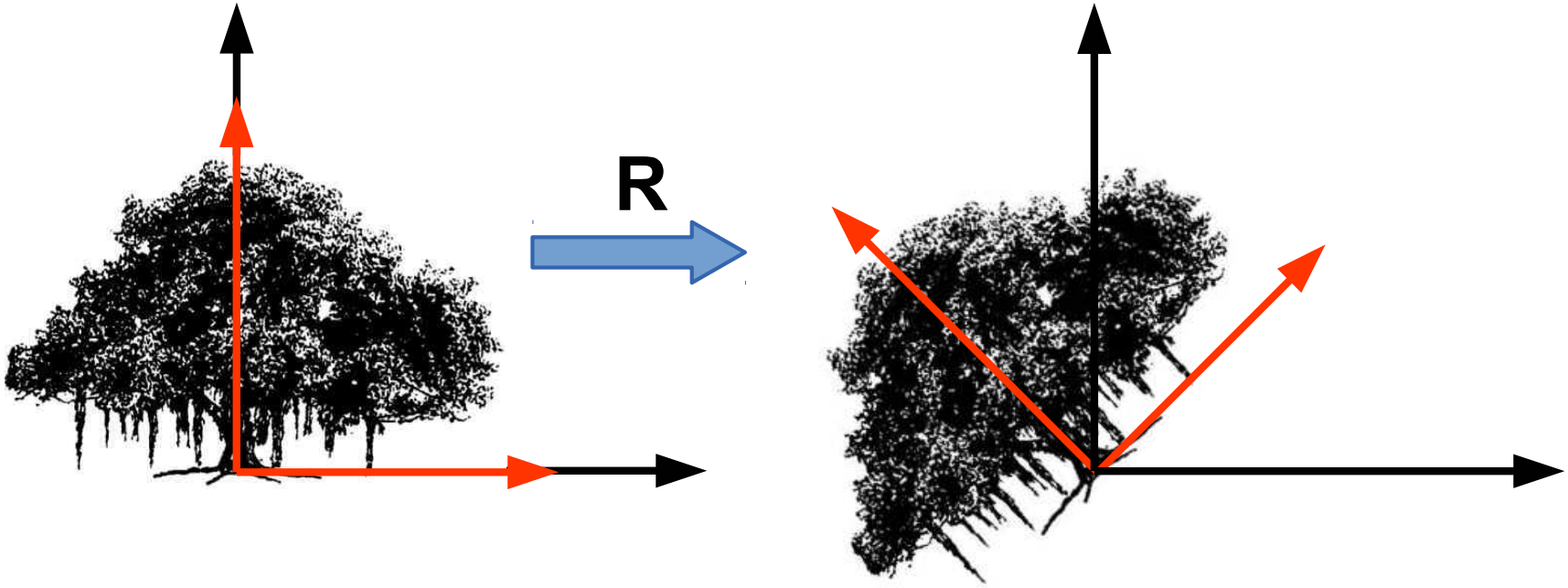
Object and Translation



$$P' = P$$

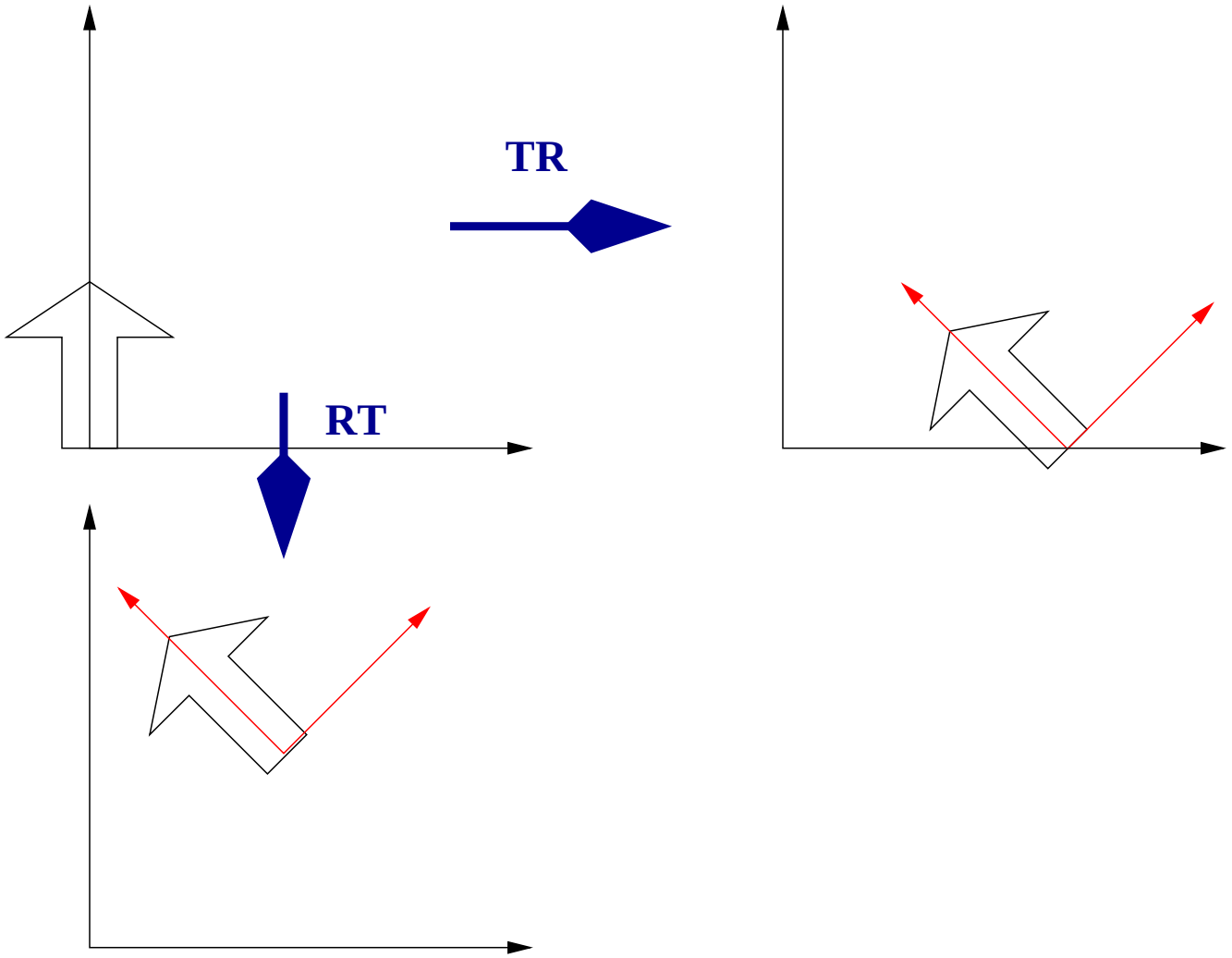
$$P' = T\ P$$

Object and Rotation



$$\mathbf{P}' = \mathbf{P} \qquad \mathbf{P}' = \mathbf{R}\ \mathbf{P}$$

Understanding Transformations

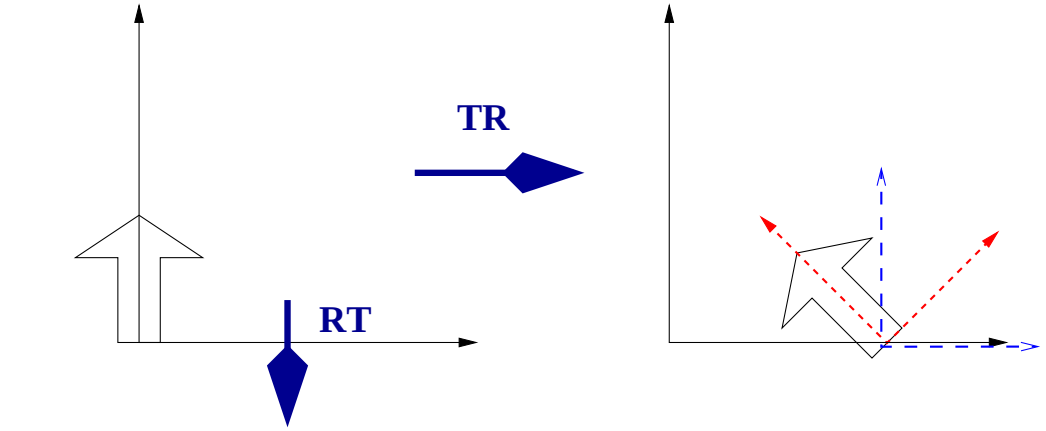


R, T: Impact on Points

- **T(5,0) R($\pi/4$)**: Impact on a point:
  - R($\pi/4$): **(Point stays at $(0, 0)$)**
  - T$(5, 0)$: **(Point goes to $(5, 0)$)**
- **R($\pi/4$) T(5,0)**: Impact on the point:
  - T$(5, 0)$: **(Point moves to $(5, 0)$)**
  - R($\pi/4$). **(Point rotates about origin)**
- All points on the shape undergo the same motions and get new coordinates

Relating Coordinate Frames

- $T(5,0)$ and $R(\pi/4)$
- Start: Black axes
  Next: Blue axes
  Last: Red axes



- $\mathbf{P}' = \overset{\textbf{Black}}{} \mid \mathbf{T} \overset{\textbf{Blue}}{\mid} \mathbf{R}\ \mathbf{P}$
- $\mathbf{P}' = \overset{\textbf{Black}}{} \mid \mathbf{R} \overset{\textbf{Blue}}{\mid} \mathbf{T}\ \mathbf{P}$

R, T: Impact on Frames

- **T(5,0) R($\pi/4$)**: Impact on coordinate frame:
  - T$(5,0)$: **(Origin shifted to $(\mathbf{5},\mathbf{0})$)**
  - R$(\pi/4)$. **(Axes rotated at new origin)**
- **R($\pi/4$) T(5,0)**: Impact on coordinate frame:
  - R$(\pi/4)$: **(Axes rotate by 45 degrees))**
  - T$(5,0)$. **(Point moves to $(\mathbf{5},\mathbf{0})$ in new axes)**
- Frames move around, giving new coordinates to points on objects!!

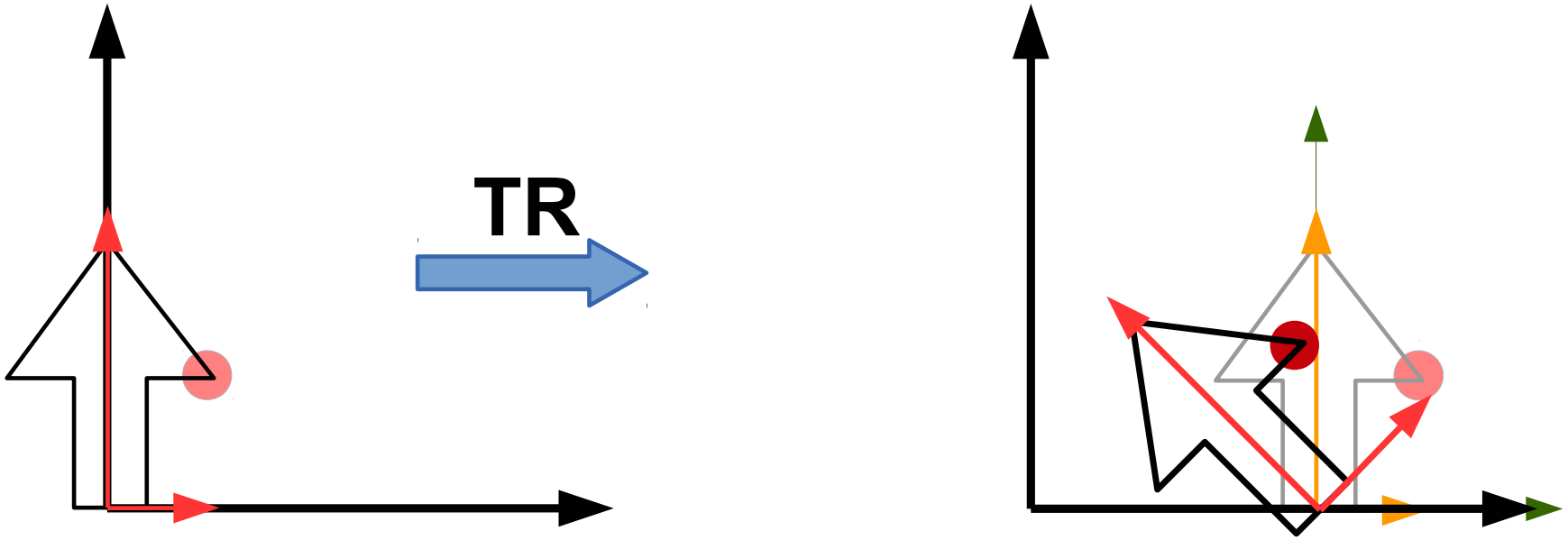
I Am Where I Think I Am (IAWITIA)

- **What am I**? Different entities at different times. student, friend/enemy, sister/brother, daughter/son, neighbour, ...
- In 3D geometry, for a point:
  What are my coordinates?
- Coordinates of a point depend on the viewpoint used (similar to life; evaluation depends on the viewer)
- Nothing really **"happens"**. Nothing **"moves"**.
  There are only changes in viewpoints (in geometry)!!
- **IAWITIA:** Pronounced ayA-wI-shia (rhymes with *dementia*)

IAWITIA in Action

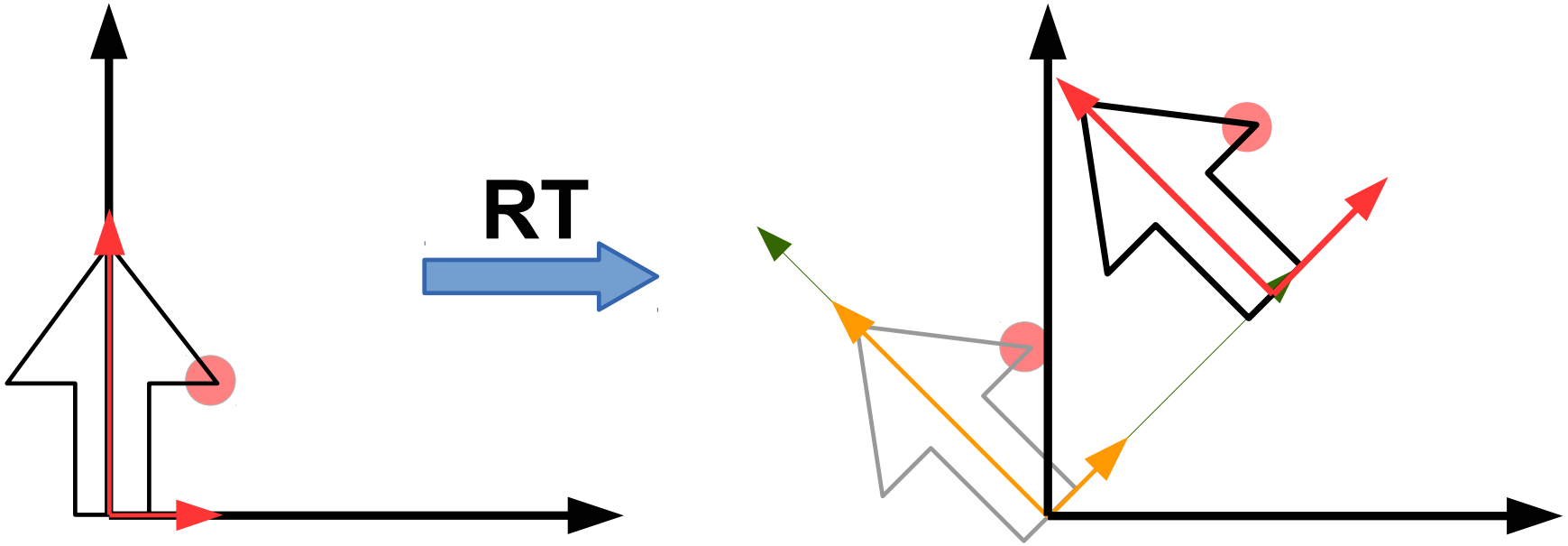
- Consider $P_4 = \mathbf{M_4}\ \mathbf{M_3}\ \mathbf{M_2}\ \mathbf{M_1}\ P_0$
- Point P_0 undergoes 4 operations and get coordinates P_4. Imagine a point having coordinates P_1, P_2, P_3 after operations $\mathbf{M_1}, \mathbf{M_2}, \mathbf{M_3}$
- Also, visualize coordinate frames $\mathbf{\Pi_4}, \mathbf{\Pi_3}, \mathbf{\Pi_2}, \mathbf{\Pi_1}, \mathbf{\Pi_0}$ in which a point has coordinates P_4 to P_0 respectively
- Operation $\mathbf{M}_i$ represents a change in coordinates from $\mathbf{\Pi}_i$ to $\mathbf{\Pi}_{i-1}$, resulting in new labels for the point.

IAWITIA in TR



- Frame translates first.
- Frame rotates next, in the *current* frame!!

IAWITIA in RT



- Frame rotates first
- Frame translates next, in the *current* frame!!

IAWITIA in IIIT Campus Model

- Model IIIT Campus as a whole. Campus is our "world"
- Global coordinate frame $\mathbf{\Pi_G}$ for the campus: at the Gate
- Buildings: Himalaya, Vindhya, Bakul, Paarijat, ..., Palash. Each has a natural coordinate frame. $\mathbf{\Pi_H}$ is Himalaya's
- Himalaya has several rooms: H105, H101, H304, etc., with own coordinate frames. $\mathbf{\Pi_C}$ is of H101 (our class)
- H101 has $\tilde{1}5$ desks, with coord frames $\mathbf{\Pi}_{Di}$ for desk i
- Desks are identical in geometry; the coord frame $\mathbf{\Pi}_{Di}$ places each in its location.

Consider Desk #13

- Consider a corner point P of desk 13, with coordinates $(200, 30, 100)$ in $\mathbf{\Pi_{D13}}$. That is: $P_{D13} = (200, 30, 100)$
- Since our world is the campus, we have to describe everything in $\mathbf{\Pi_G}$

$$P_G = \mathbf{M_{GH}}\, \mathbf{M_{HC}}\, \mathbf{M_{CD13}}\; P_{D13}$$

- $\mathbf{M_{GH}}$ aligns $\mathbf{\Pi_G}$ to $\mathbf{\Pi_H}$. $\mathbf{M_{HC}}$ aligns $\mathbf{\Pi_H}$ to $\mathbf{\Pi_C}$. $\mathbf{M_{CD13}}$ aligns $\mathbf{\Pi_C}$ to $\mathbf{\Pi_{D13}}$
- $P_G = \mathbf{M_{GH}} \overset{P_H}{|}\ \mathbf{M_{HC}} \overset{P_C}{|}\ \mathbf{M_{CD13}} \overset{P_{D13}}{|}\ P$ (for any point P on Desk13)
- We can place a given desk in any **building, room, place**!

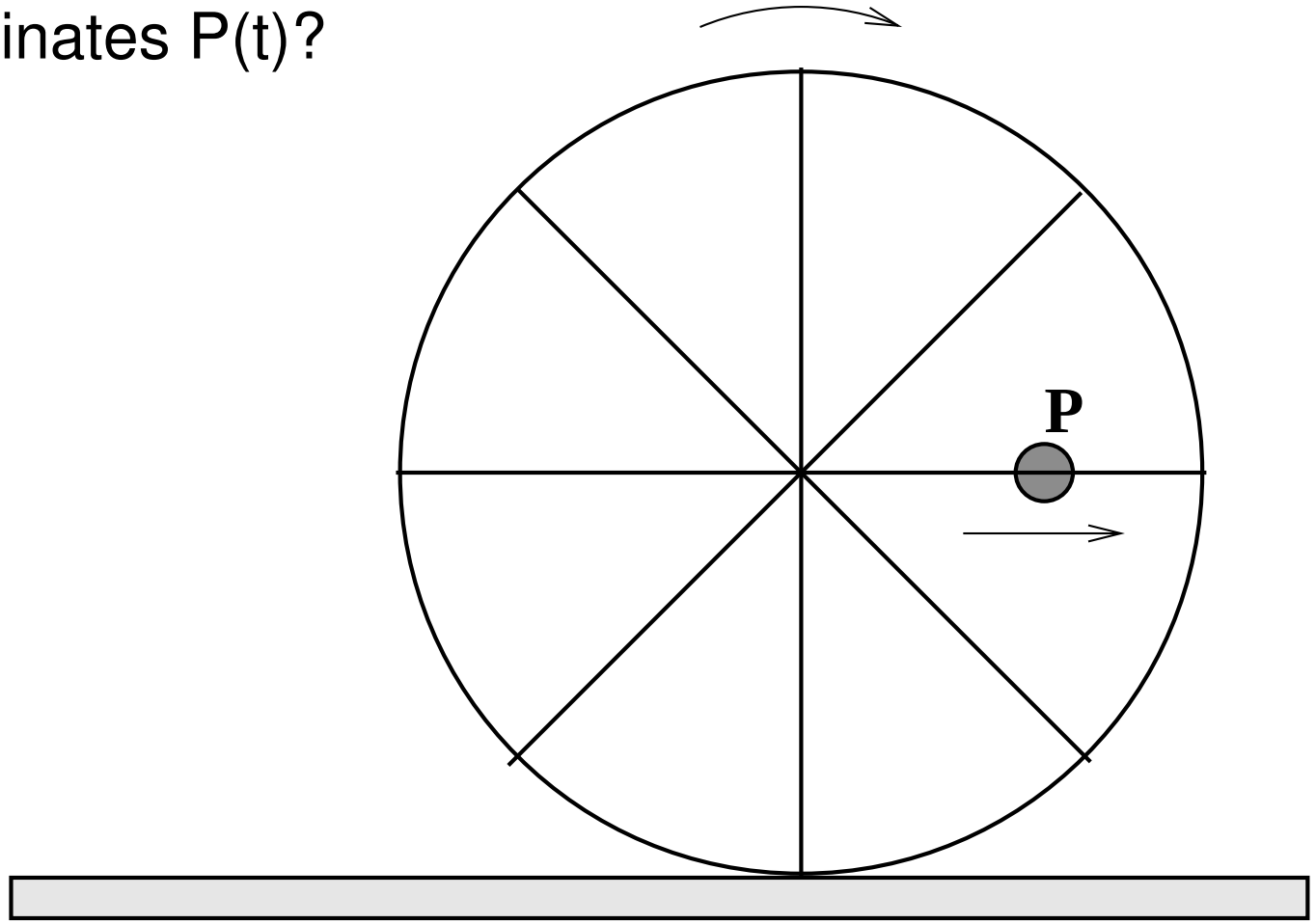
Walking on Stage

- Teacher walks horizontally on stage, with swinging arms
- The hand-tip moves w.r.t each student? **How?**
- Student knows own position in room's reference frame
- Start at a student's eye. Provides the viewpoint. Align to room's reference frame using $\mathbf{M_{SR}}$. Different for each student. But at the room now!
- Walk: pure translation. $\mathbf{M_{RT}}$ aligns to teacher frame.
- Arm swing: Simple harmonic motion with angle $\theta(t)$
- ....

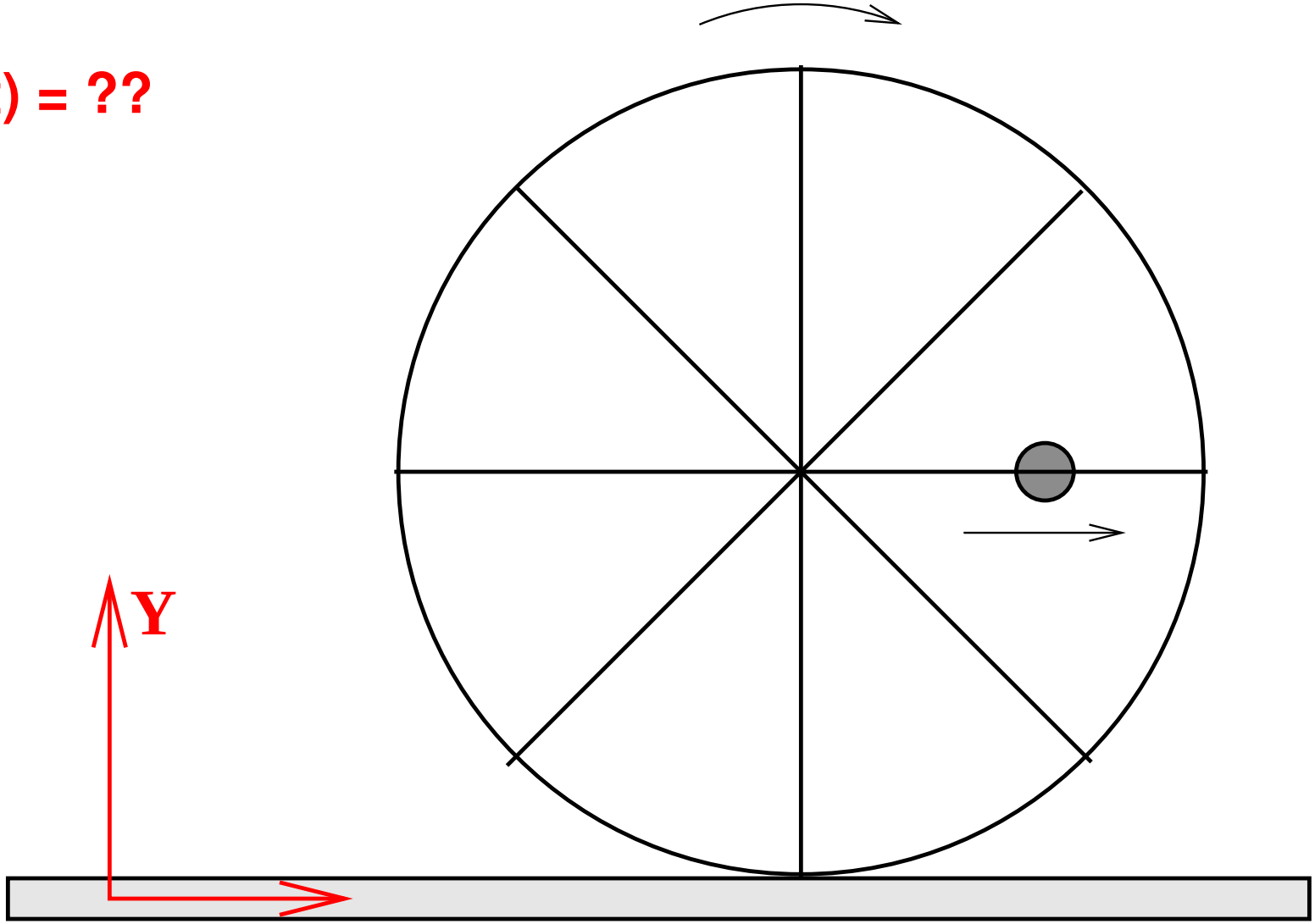
Simpler situation in each new coord frames. **IAWITIA**!!

Rolling Wheel

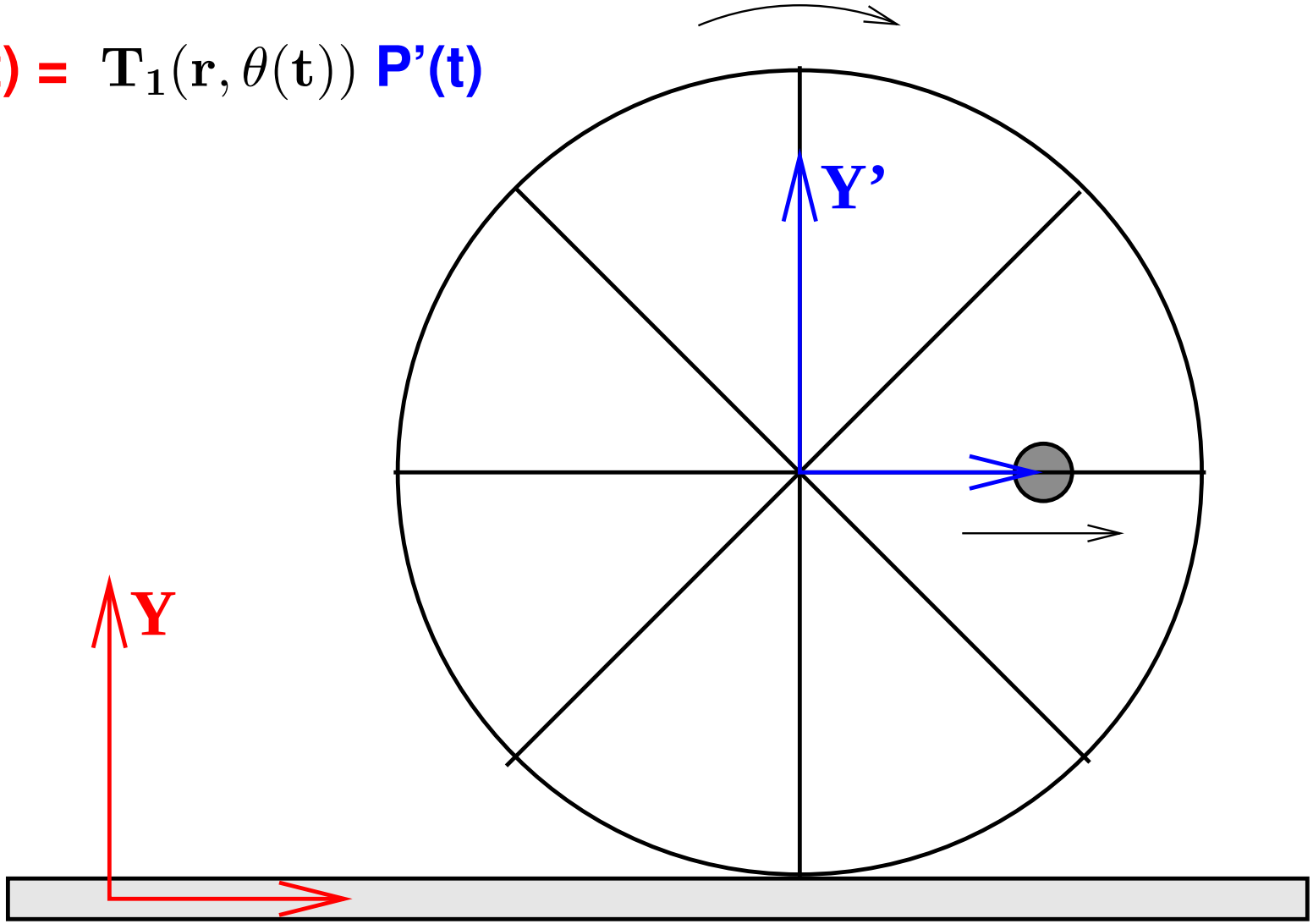
Coordinates P(t)?



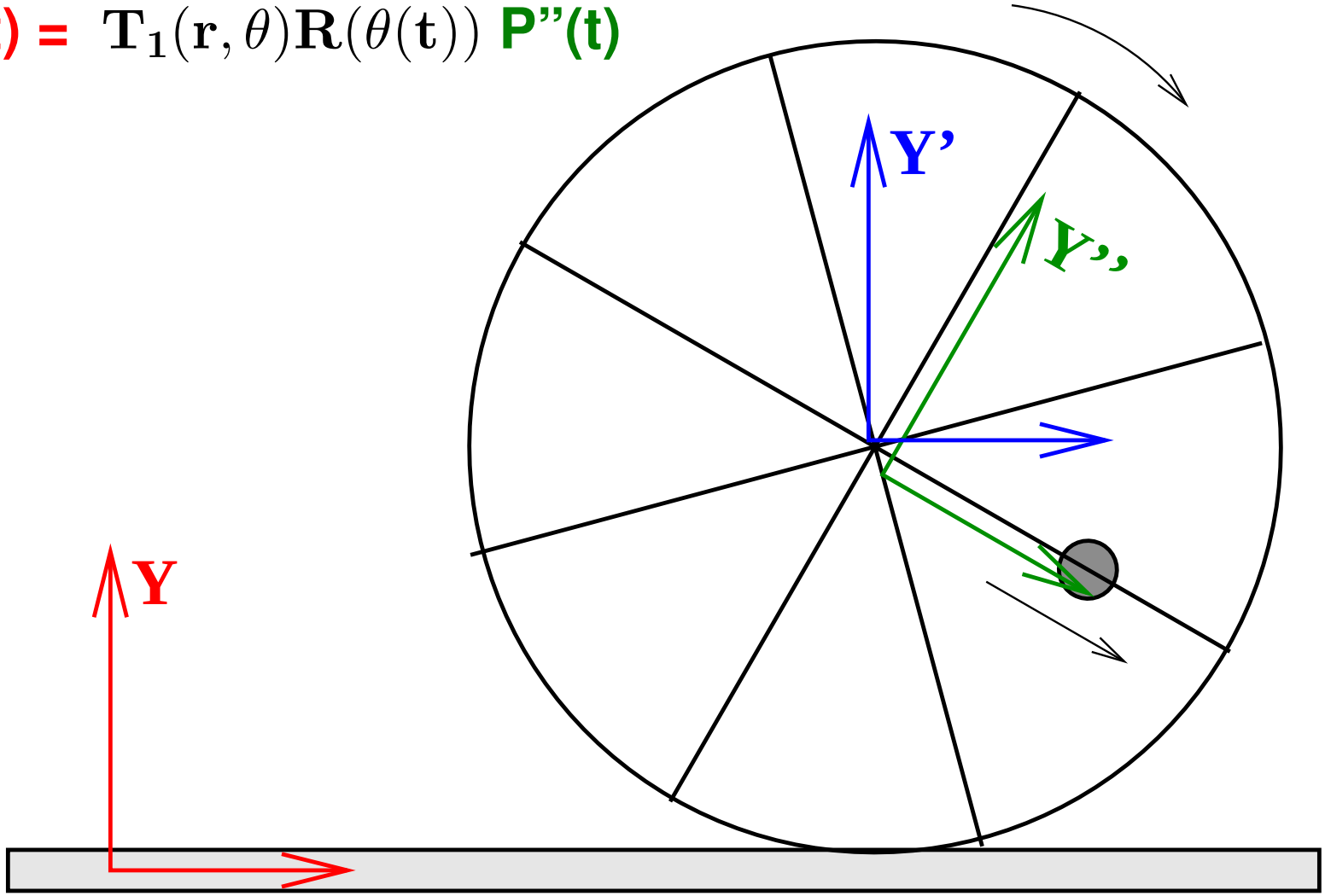
P(t) = ??



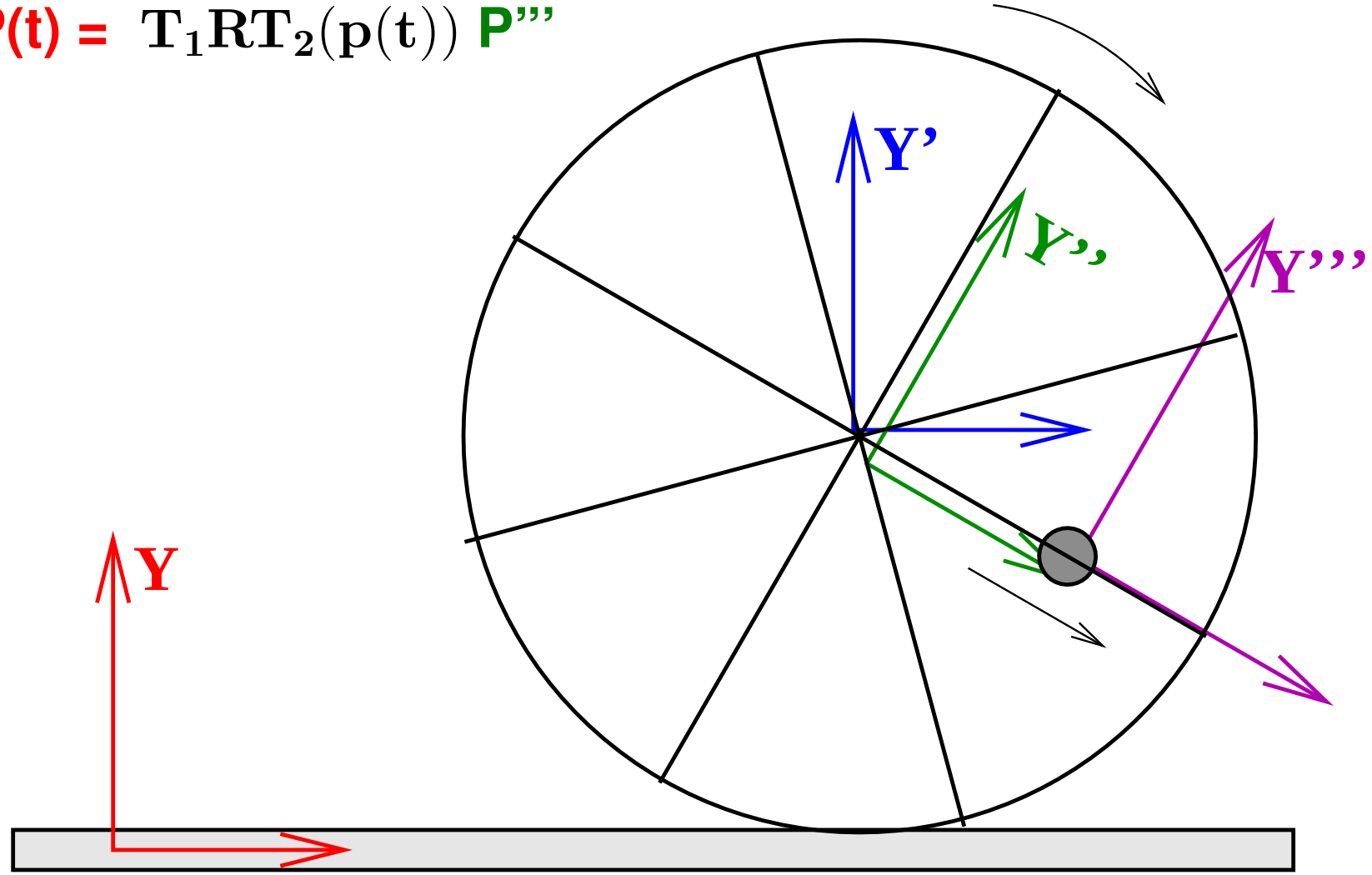
P(t) = $\mathbf{T}_1(\mathbf{r}, \theta(\mathbf{t}))$ **P'(t)**



P(t) = $\mathbf{T}_1(\mathbf{r}, \theta)\mathbf{R}(\theta(\mathbf{t}))$ **P”(t)**



P(t) = $\mathbf{T_1 R T_2(p(t))}$ **P'''**

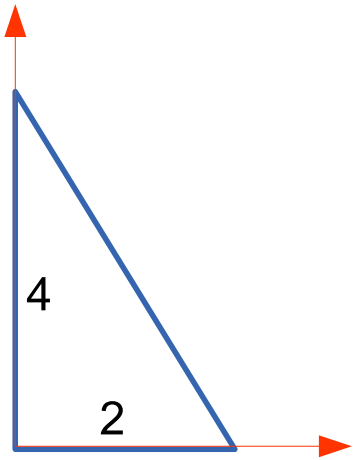


Final Transformation

- **P(t) =** $\mathbf{T_1(t)\ R(\theta(t))\ T_2(p(t))}$ **P'''**
- $\mathbf{T_1(t) = T(r\ \theta(t), 0) = T(r\ \omega\ t, 0)}$ (A translation matrix)
- $\mathbf{R(\theta(t)) = R_Z(\omega\ t)}$ (A normal rotation matrix)
- $\mathbf{T_2(t) = T(p(t), 0) = T(v\ t, 0)}$ (A translation matrix)
- **P'''** = $[0, 0, 1]^T$ (Origin of the bead)
- Lot simpler than thinking about it all together.
- What if we have a pendulum swinging freely on the bead?

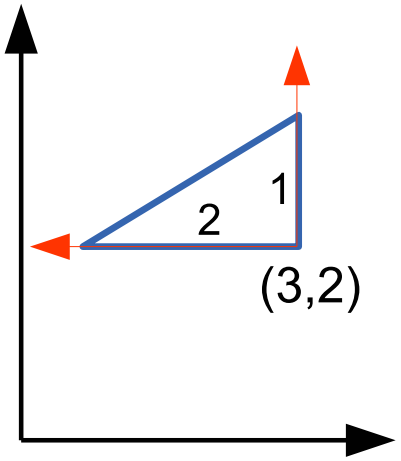
Given an object

- An object `traingleObj` is given. Can be drawn using `drawObject(triangleObj)`



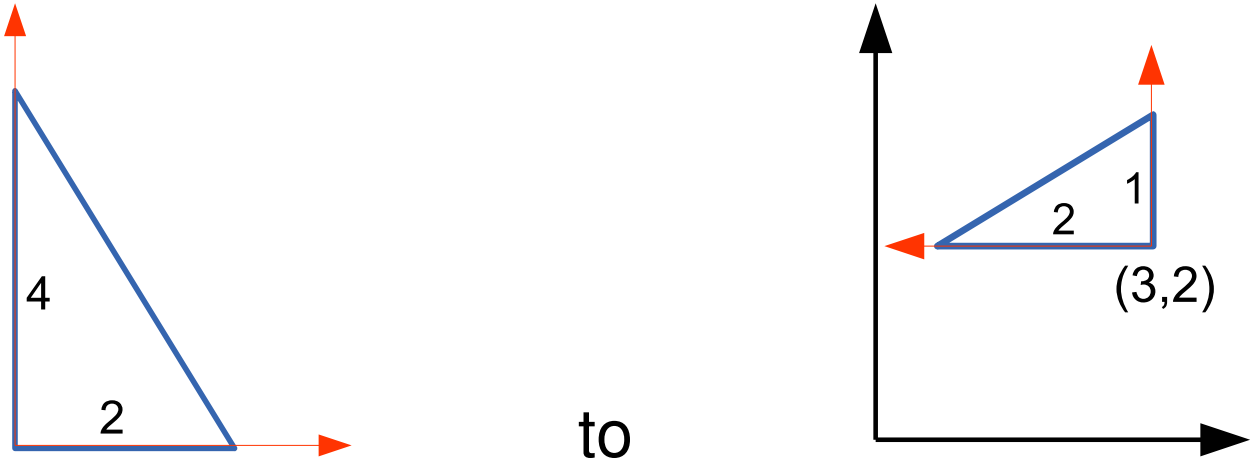
- `drawObject(triangleObj)` draws the object at (current) origin

Draw it in a different configuration



- Use `drawObject(triangleObj)`, with right transformations

Transformations

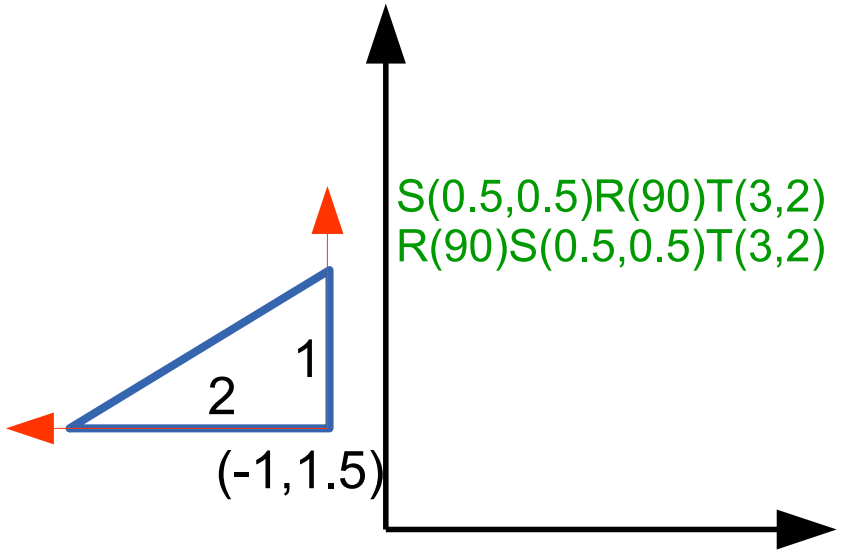


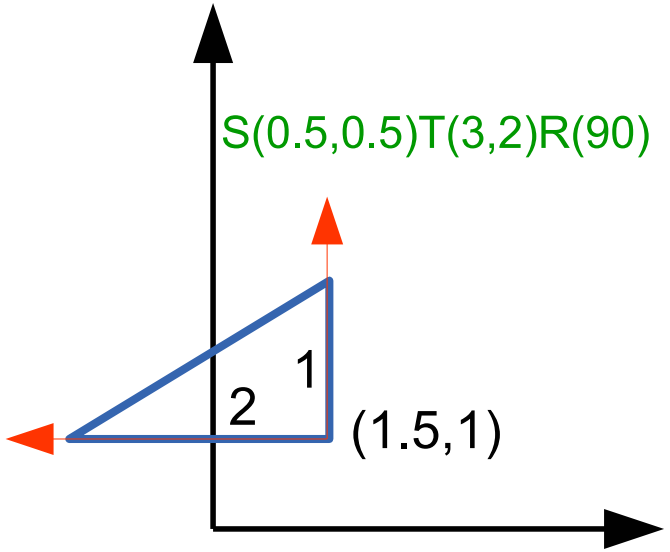
- What transformations? Translation, Rotation, Scaling!!
  – Specifically: $\mathbf{S}(\frac{\mathbf{1}}{\mathbf{2}}, \frac{\mathbf{1}}{\mathbf{2}}), \mathbf{T}(\mathbf{3}, \mathbf{2}), \mathbf{R}(\mathbf{90})$
- What happens with $S(0.5, 0.5)$? Axes "shrink"; unit length shrinks! New measurements have new values!

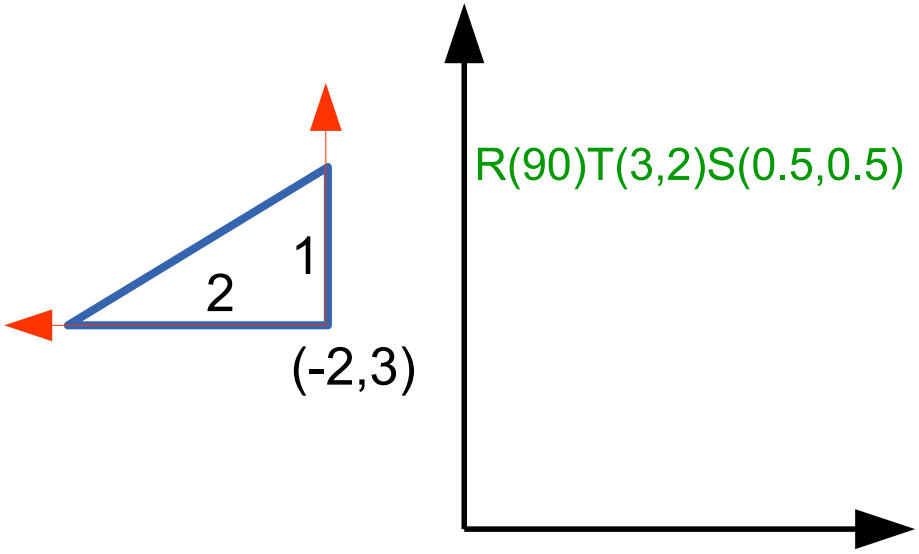
Which combination?

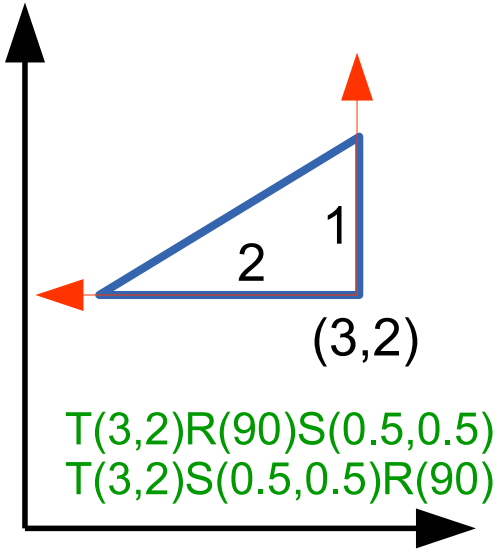
Envision and sketch the impact of each of:

1. $\mathbf{S}(\frac{1}{2}, \frac{1}{2})\ \mathbf{R}(90)\ \mathbf{T}(3, 2)$
2. $\mathbf{S}(\frac{1}{2}, \frac{1}{2})\ \mathbf{T}(3, 2)\ \mathbf{R}(90)$
3. $\mathbf{T}(3, 2)\ \mathbf{R}(90)\ \mathbf{S}(\frac{1}{2}, \frac{1}{2})$
4. $\mathbf{T}(3, 2)\ \mathbf{S}(\frac{1}{2}, \frac{1}{2})\ \mathbf{R}(90)$
5. $\mathbf{R}(90)\ \mathbf{S}(\frac{1}{2}, \frac{1}{2})\ \mathbf{T}(3, 2)$
6. $\mathbf{R}(90)\ \mathbf{T}(3, 2)\ \mathbf{S}(\frac{1}{2}, \frac{1}{2})$


S(0.5,0.5)R(90)T(3,2)
R(90)S(0.5,0.5)T(3,2)
2
1
(-1,1.5)

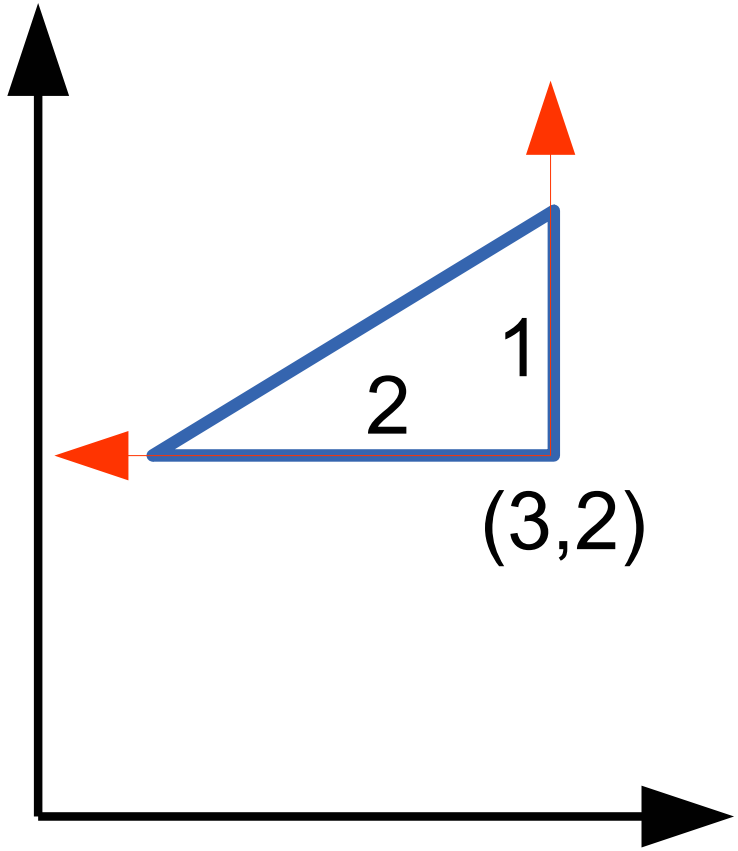

S(0.5,0.5)T(3,2)R(90)
2
1
(1.5,1)


R(90)T(3,2)S(0.5,0.5)
2
1
(-2,3)


2
1
(3,2)
T(3,2)R(90)S(0.5,0.5)
T(3,2)S(0.5,0.5)R(90)

Several Correct Situations

T(3,2)R(90)S(0.5,0.5)
T(3,2)S(0.5,0.5)R(90)
R(90)T(2,-3)S(0.5,0.5)
R(90)S(0.5,0.5)T(4,-6)
S(0.5,0.5)R(90)T(4,-6)
S(0.5,0.5)T(6,4)R(90)



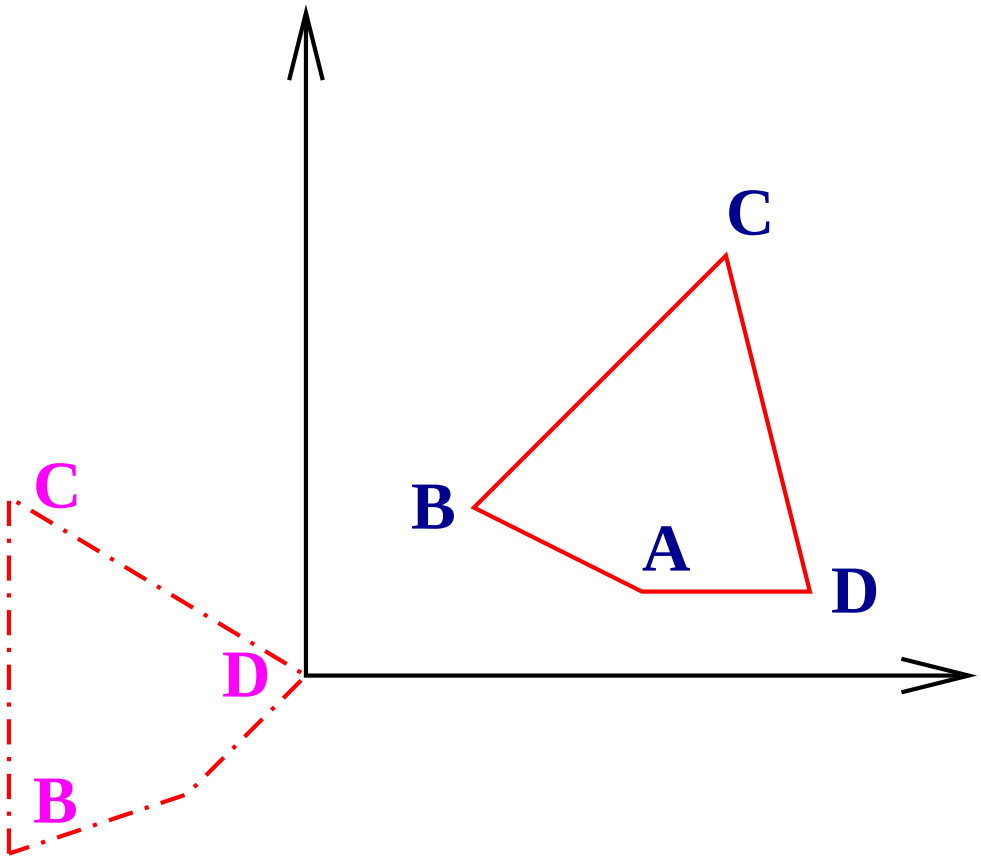
Another Situation

- A clock is hanging from a nail fixed to a flat plate. The plate is being translated with a velocity $\vec{\mathbf{v}}$ and acceleration $\vec{\mathbf{a}}$. The pendulum of the clock swings back and forth with a time period of 5 seconds and a max angle of $\pm\theta$. An ant travels from the bottom tip of the pendulum up to the centre.

- How do we compute the ant's position with respect to a fixed coordinate system coplanar with the plate?

  Please sketch the situation and work it out for yourself

A Transformation Problem



Bring **D** to origin and **BC** parallel to the Y axis as shown

Transformation Computation

- Step 1: Translate by $-\mathbf{D}$. What is the orientation of BC?
- Step 2: Rotate to have unit vector $\vec{\mathbf{u}} = [u_x \;\; u_y]^\intercal$ from **B** to **C** on the Y axis. That is the second row of **R** matrix
- The matrix for the total operation: $\mathbf{M} = \mathbf{R}\,\mathbf{T}(-\mathbf{D})$
- Two options for first row. $[u_y \;\; -u_x]^\intercal$ and $[-u_y \; u_x]^\intercal$
- **R** matrix: (a) $\begin{bmatrix} u_y & -u_x \\ u_x & u_y \end{bmatrix}$ or (b) $\begin{bmatrix} -u_y & u_x \\ u_x & u_y \end{bmatrix}$?
- Difference? The direction aligned to the X-axis!
- Option (a) is correct. **Why?** Draw Option (b)!

Easy 3D Transformations

$$T(a,b,c) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(a,b,c) = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- CCW +ve; orthonormal; length preserving; rows give direction vectors that rotate onto each axis; columns ....

Rotation about an axis parallel to Z

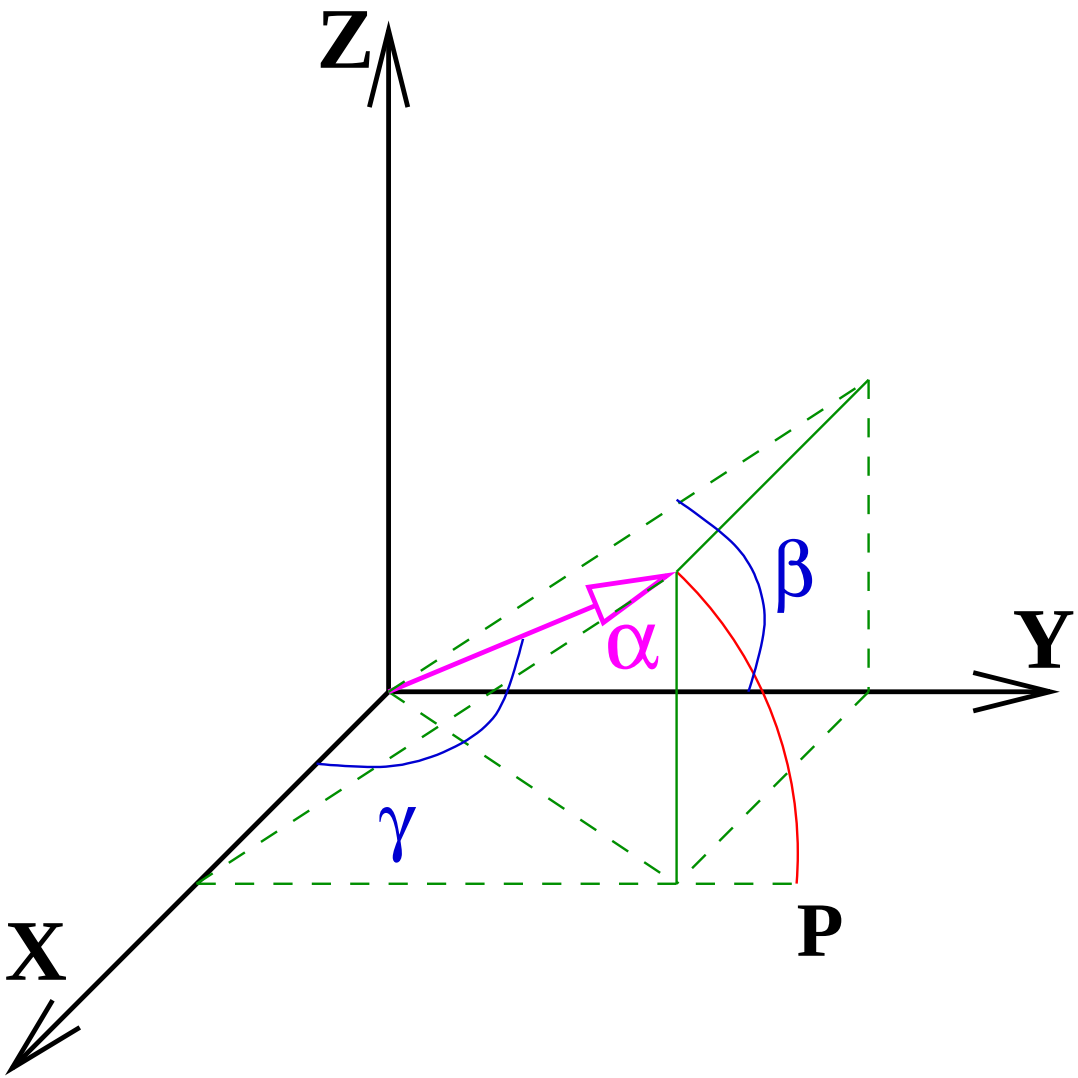
- An axis parallel to Z axis, passing through point $(x, y, 0)$.
- Translate so that the axis passes through the origin: $\mathbf{T}(-x, -y, k)$ for any k!!
- Overall: $\mathbf{M} = \mathbf{T}(x, y, -k)\ \mathbf{R_Z}(\theta)\ \mathbf{T}(-x, -y, k)$
- Why shouldn't k matter? $\mathbf{R_Z}$ doesn't affect the z coordinate. So, whatever k is added first will be subtracted later

3D Rotation about an axis α

- What is $\mathbf{R}_{\alpha}(\theta)$?
- How do we reduce it to something we know?
- What do we know? $\mathbf{R}_{\mathbf{X}}(\theta), \mathbf{R}_{\mathbf{Y}}(\theta), \mathbf{R}_{\mathbf{Z}}(\theta)$

3D Rotation about an axis α

- What is $\mathbf{R}_{\alpha}(\theta)$?
- 3-step process:
  1. Apply $\mathbf{R}_{\alpha\mathbf{x}}$ to align α with the X axis.
  2. Rotate about X by angle θ.
  3. Undo the first rotation using $\mathbf{R}_{\alpha\mathbf{x}}^{-1}$
- Net result: $\mathbf{R}_{\alpha}(\theta) \;=\; \mathbf{R}_{\alpha\mathbf{x}}^{-1}\,\mathbf{R}_{\mathbf{x}}(\theta)\,\mathbf{R}_{\alpha\mathbf{x}}$
- Quite simple!? What is $\mathbf{R}_{\alpha\mathbf{x}}(\theta)$?
- **(We can align α with Y or Z axis also)**


Z
Y
X
P
α
β
γ

Computing $\mathbf{R}_\alpha$

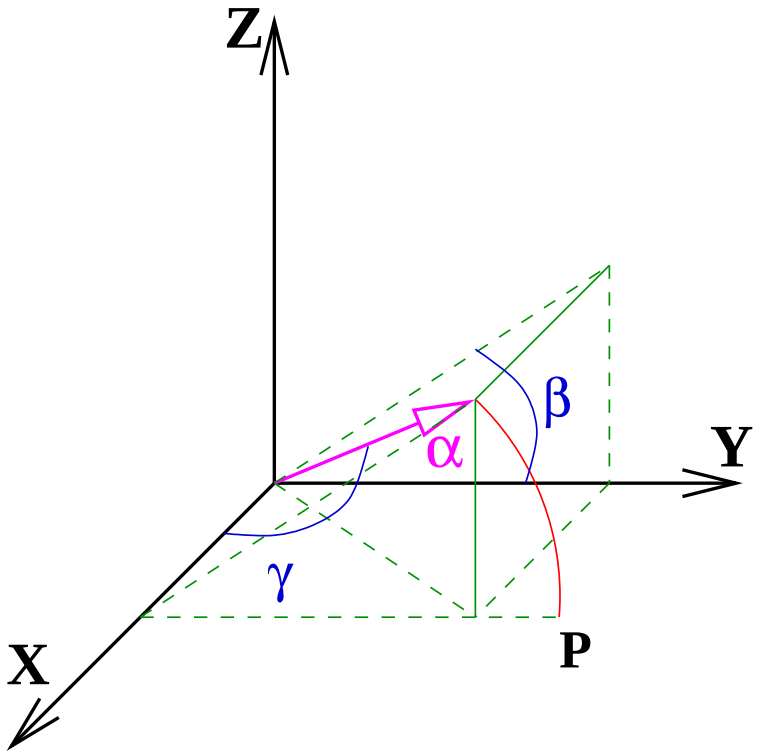
- First rotate by $-\beta$ about X axis. Vector α would lie in the XY plane, with tip at point **P**.
- $\beta = ?, \quad \tan\beta = ?$
- Next rotate by $-\gamma$ about Z axis. Vector α would coincide with the X axis.
- $\gamma = ?, \quad \tan\gamma = ?$

Computing $\mathbf{R}_\alpha$

- Rotate by $-\beta$ about X axis to bring α to XY plane
- $\tan\beta = \frac{\alpha_z}{\alpha_y}$
- Rotate by $-\gamma$ about Z axis to bring α to X axis
- $\tan\gamma = \frac{\sqrt{\alpha_y^2+\alpha_z^2}}{\alpha_x} = \frac{\sqrt{1-\alpha_x^2}}{\alpha_x}$ if $|\alpha| = 1$.
- $\mathbf{R}_{\alpha\mathbf{x}} = \mathbf{R_z}(-\gamma)\mathbf{R_x}(-\beta)$ and $\mathbf{R}_{\alpha\mathbf{x}}^{-\mathbf{1}} = \mathbf{R_x}(\beta)\mathbf{R_z}(\gamma)$
- Alternative: Don't we know about **rotation matrices** and direction cosines that go **to/from coordinate axes**?

Final

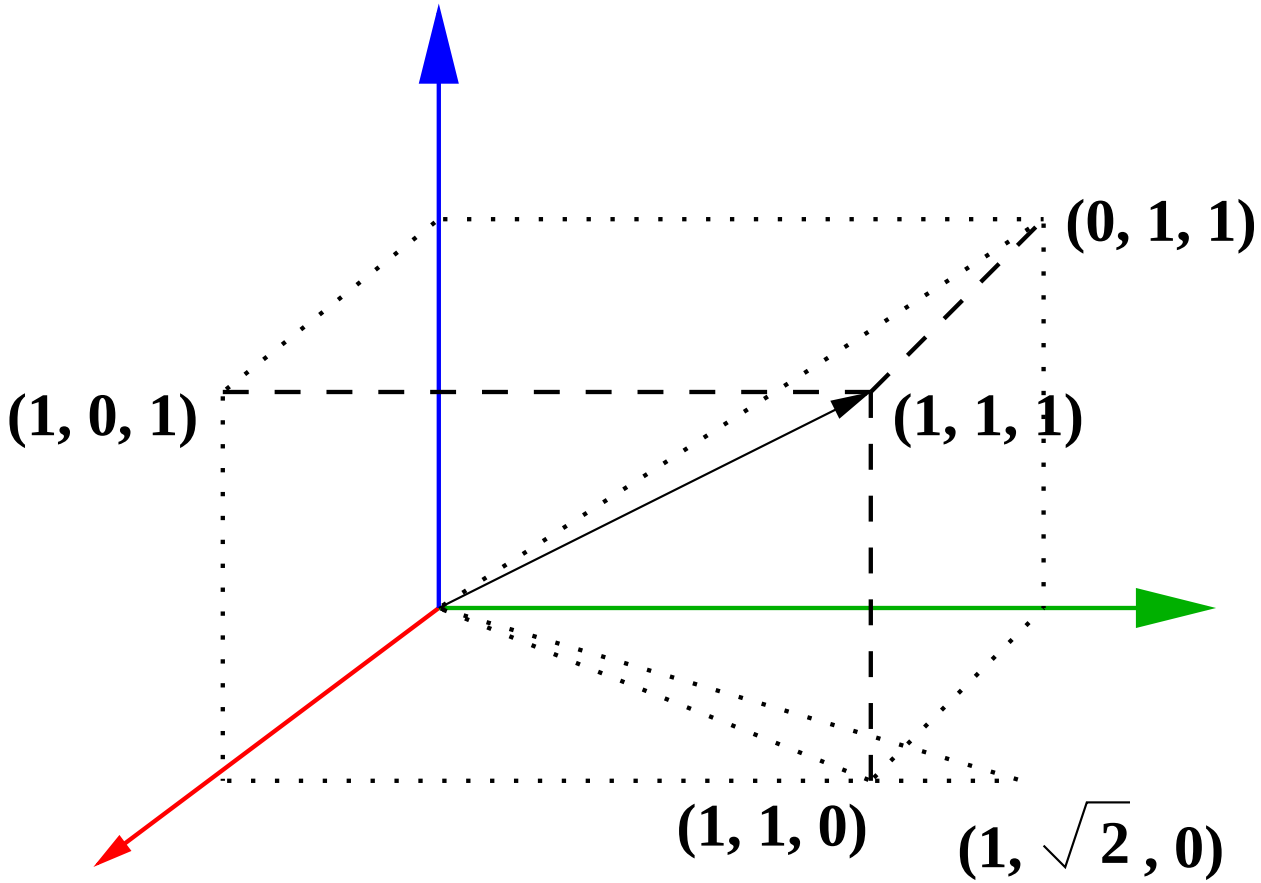
- $\mathbf{R}_\alpha(\theta) = \mathbf{R_x}(\beta)\mathbf{R_z}(\gamma) \quad \mathbf{R_x}(\theta) \quad \mathbf{R_z}(-\gamma)\mathbf{R_x}(-\beta)$



Alternate $\mathbf{R}_{\alpha \mathbf{x}}$

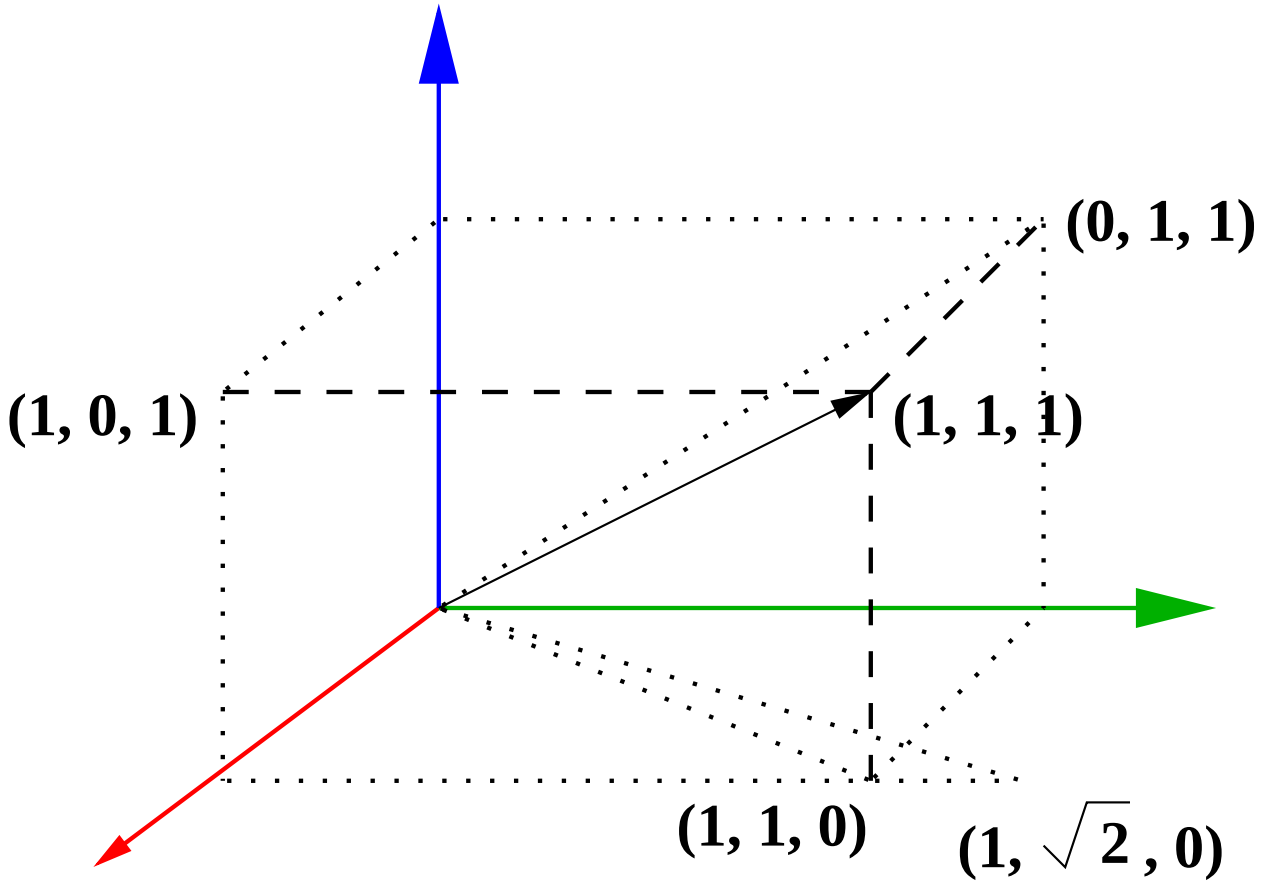
- After rotation, α will align with X-axis. Hence that is the first row $\mathbf{r_1}$ of the rotation matrix
- Find a direction orthogonal to α to be row $\mathbf{r_2}$. How?
- Take any vector $\mathbf{v}$ not parallel to α. $\mathbf{r_2} = \alpha \times \mathbf{v}$ will work!!
- Lastly, $\mathbf{r_3} = \mathbf{r_1} \times \mathbf{r_2}$ and $\mathbf{R}_{\alpha \mathbf{x}} = \begin{bmatrix} \alpha & 0 \\ \alpha \times \mathbf{v} & 0 \\ \mathbf{r_1} \times \mathbf{r_2} & 0 \\ \mathbf{0} & 1 \end{bmatrix}$
- Many possibilities, all with the same result (hopefully...)

Example: Rotation about $[1\ 1\ 1]^{\mathrm{T}}$



$\beta = ?, \qquad \gamma = ?$

Example: Rotation about $[1\ 1\ 1]^{\mathrm{T}}$



$\tan\beta = 1, \qquad \tan\gamma = \sqrt{2}$

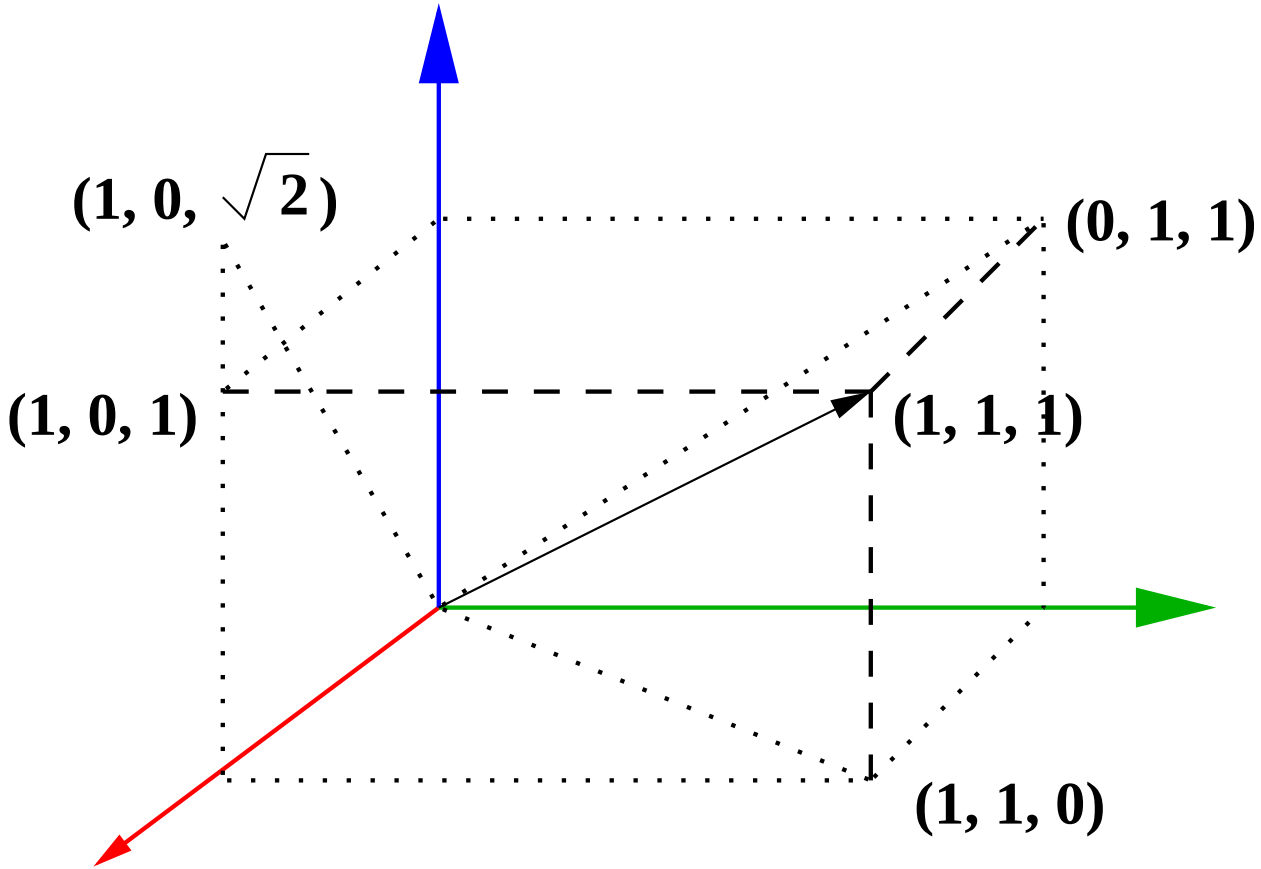
Computing $\mathbf{R}_{\alpha\mathbf{x}}$: Method 1

- Rotate by $-\pi/4$ about X. $\mathbf{R_X}(-\frac{\pi}{4}) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{-1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$
- $\mathbf{R_Z}(-\arctan(\sqrt{\mathbf{2}})) = \begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{2}{\sqrt{6}} & 0 \\ \frac{-2}{\sqrt{6}} & \frac{1}{\sqrt{3}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$
- $\mathbf{R}^{\mathbf{I}}_{\alpha\mathbf{x}} = \mathbf{R_Z}(-\tan^{-1}(\sqrt{\mathbf{2}}))\ \mathbf{R_X}(-\frac{\pi}{4}) = \begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & 0 \\ \frac{-2}{\sqrt{6}} & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} & 0 \\ 0 & \frac{-1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Computing $\mathbf{R}_{\alpha\mathbf{x}}$: Method 2

- $[\mathbf{1}\ \mathbf{1}\ \mathbf{1}]^{\mathbf{T}}$ will lie on X-axis. First row $\mathbf{r_1} = [\frac{\mathbf{1}}{\sqrt{\mathbf{3}}}\ \ \frac{\mathbf{1}}{\sqrt{\mathbf{3}}}\ \ \frac{\mathbf{1}}{\sqrt{\mathbf{3}}}]^{\mathbf{T}}$.
- Second row: $\mathbf{r_2} = \alpha \times [\mathbf{1}\ \mathbf{0}\ \mathbf{0}]^{\mathbf{T}} = [\mathbf{0}\ \ \frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\ \ \frac{\mathbf{-1}}{\sqrt{\mathbf{2}}}]^{\mathbf{T}}$
- Third row: $\mathbf{r_3} = \alpha \times [\mathbf{0}\ \ \frac{\mathbf{1}}{\sqrt{\mathbf{2}}}\ \ \frac{\mathbf{-1}}{\sqrt{\mathbf{2}}}]^{\mathbf{T}} = [\frac{\mathbf{2}}{\sqrt{\mathbf{6}}}\ \ \frac{\mathbf{-1}}{\sqrt{\mathbf{6}}}\ \ \frac{\mathbf{-1}}{\sqrt{\mathbf{6}}}]^{\mathbf{T}}$
- $\mathbf{R}^{\mathbf{II}}_{\alpha\mathbf{x}} = \begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & 0 \\ 0 & \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{6}} & \frac{-1}{\sqrt{6}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \mathbf{R_Y}(\tan^{-1}(\sqrt{2}))\ \mathbf{R_X}(\frac{\pi}{4})$

$\mathbf{R}_{\alpha x}$ Method 2: Contd



Question: Which vector v yields the matrix of Method 1?

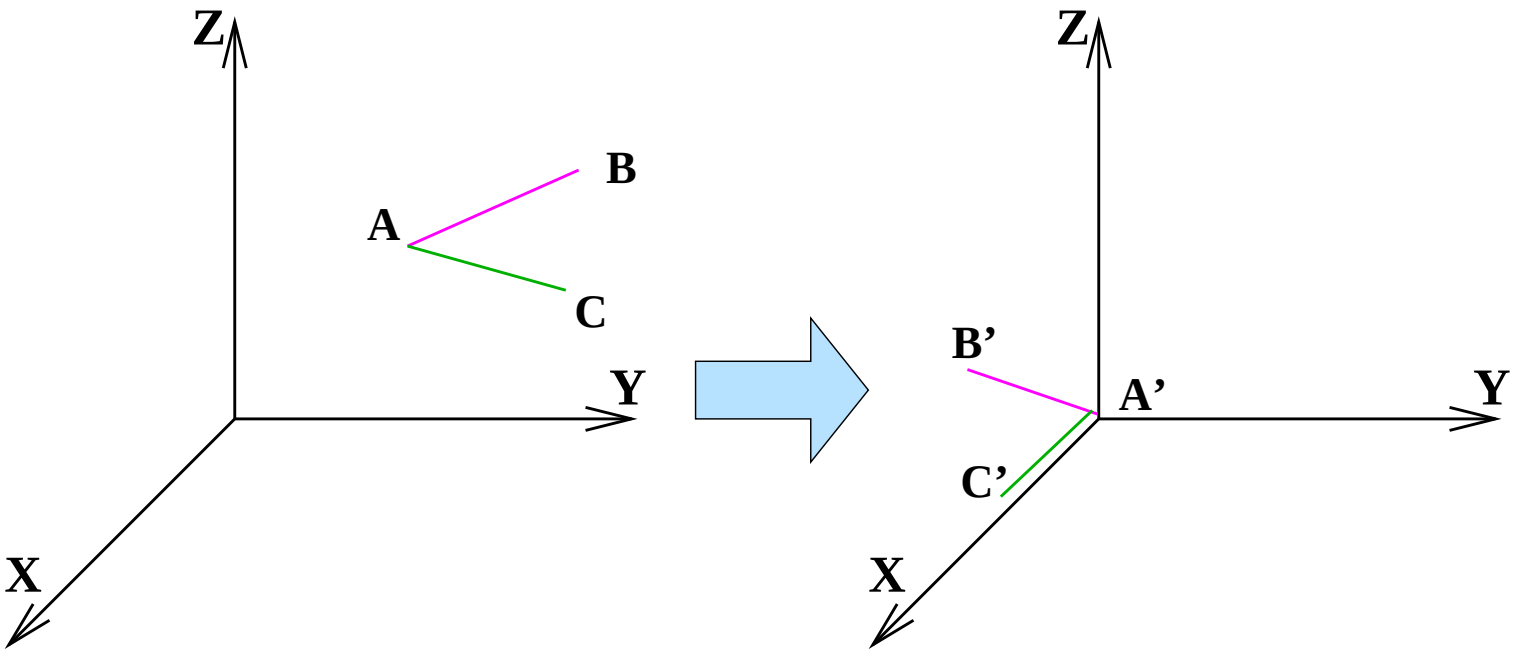
Rotation: Arbitrary Axis, Point

- An arbitrary axis may not pass through the origin.
- Translate by $\mathbf{T}$ so that it passes through the origin.
- Apply $\mathbf{R}_\alpha$.
- Translate back using $\mathbf{T}^{-1}$.
- Composite transformation: $\mathbf{T}^{-1}\,\mathbf{R}_\alpha\,\mathbf{T}$.

3D Transformations

- Many ways to *think about* a given transform.
- Ultimately, there is only one transform given the starting and ending configurations.
- Different methods let us analyze the problem from different perspectives.

Another Example



Working the Example

- Translate by **-A** to bring it to the origin.
- After the rotation, **AC** sits on the X axis.
- The first row of the rotation matrix is **AC** / |**AC**|.
- The vector normal to the plane **ABC** sits on the Y axis.
- The second row of the rotation matrix is the unit vector along **AB** $\times$ **AC = (AB** $\times$ **AC)** / |**AB** $\times$ **AC**|.
- Third row is a cross product of the first two.
- Final transformation: $\mathbf{R}\,\mathbf{T}(-\mathbf{A})$

Transforming Lines

- A composite transformation can be seen as changing points in the coordinate system.
- These transformations preserve collinearity. Thus, points on a line remain on a (transformed) line.
- Take two points on the line, transform them, and compute the line between new points.
- Lines are defined as a join of 2 points or intersection of 2 planes in 3D. The same holds for transformed lines using transformed points or planes!

Transforming Planes

- A plane is defined by a 4-vector $\mathbf{n}$ (called the **normal** vector) in homogeneous coordinates.
- The plane consists of points $\mathbf{p}$ such that $\mathbf{n}^\intercal\mathbf{p} = \mathbf{0}$.
- Let $\mathbf{n}$ go to $\mathbf{Q}\,\mathbf{n}$ when points are transformed by $\mathbf{M}$.
- Coplanarity is preserved: $(\mathbf{Qn})^\intercal\mathbf{Mp} = \mathbf{0} = \mathbf{n}^\intercal\mathbf{Q}^\intercal\mathbf{Mp}$.
- True when $\mathbf{Q}^\intercal\mathbf{M} = \mathbf{I}$, or $\mathbf{Q} = \mathbf{M}^{-\intercal}$.
- $\mathbf{Q}$ is the Matrix of cofactors of $\mathbf{M}$ in the general case when $\mathbf{M}^{-1}$ doesn't exist.

Understanding Geometric Transformations

- Geometry transformation of objects is very important to compose graphics environments
- Understand what you want to be achieved, visualize it in your mind and compose the series of transformations
- Needs getting used to the ideas. Think about getting into a simpler situtation from the current one.

How do we use in Graphics/OpenGL

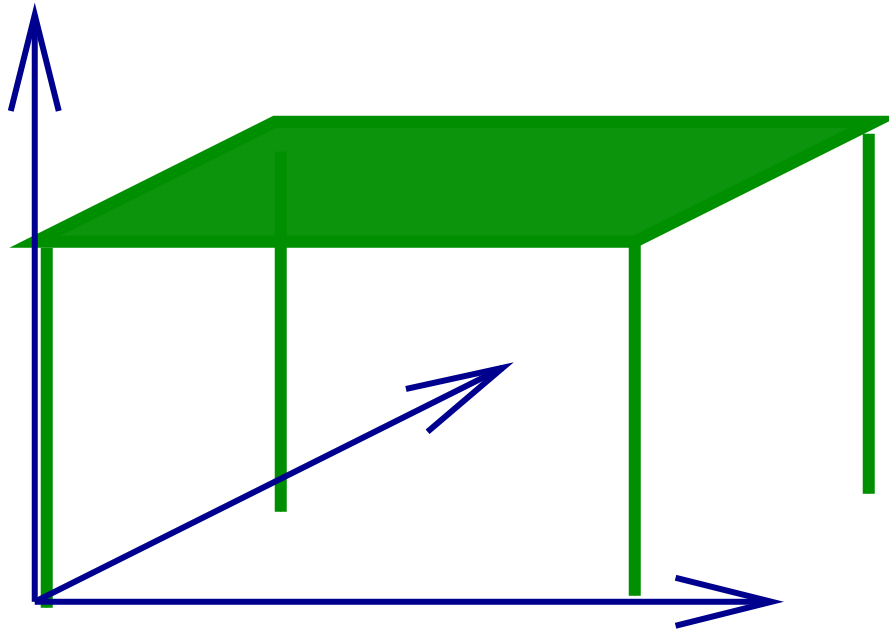
- We set up the "world" using multiple different objects
  – Transformations to place and orient each object
- Place camera also in the world coordinate system
  – Camera has a location and an orientation (6 DoF)
- What matters: **How does the world look from the camera's point of view?**
  – Camera: leftmost coord frame. Object: rightmost!
- $P_C = \mathbf{M_{CW}}\,\mathbf{M_{WO}}\;P_O$.
  P_C: Camera coordinates, P_O: Object coordinates.
  $\mathbf{M_{CW}}$: Aligns Camera frame to World frame.
  $\mathbf{M_{WO}}$: Aligns World frame to Object frame.

How do we draw multiple objects?

- $P_C = \mathbf{M_{CW}} \overset{P_W}{|} \mathbf{M_{WO^1}} \overset{P_{O^1}}{|} P$ (for points of Object 1)
- $P_C = \mathbf{M_{CW}} \overset{P_W}{|} \mathbf{M_{WO^2}} \overset{P_{O^2}}{|} P$ (for points of Object 2)
- $P_C = \mathbf{M_{CW}} \overset{P_W}{|} \mathbf{M_{WO^i}} \overset{P_{O^i}}{|} P$ (for points of Object i)

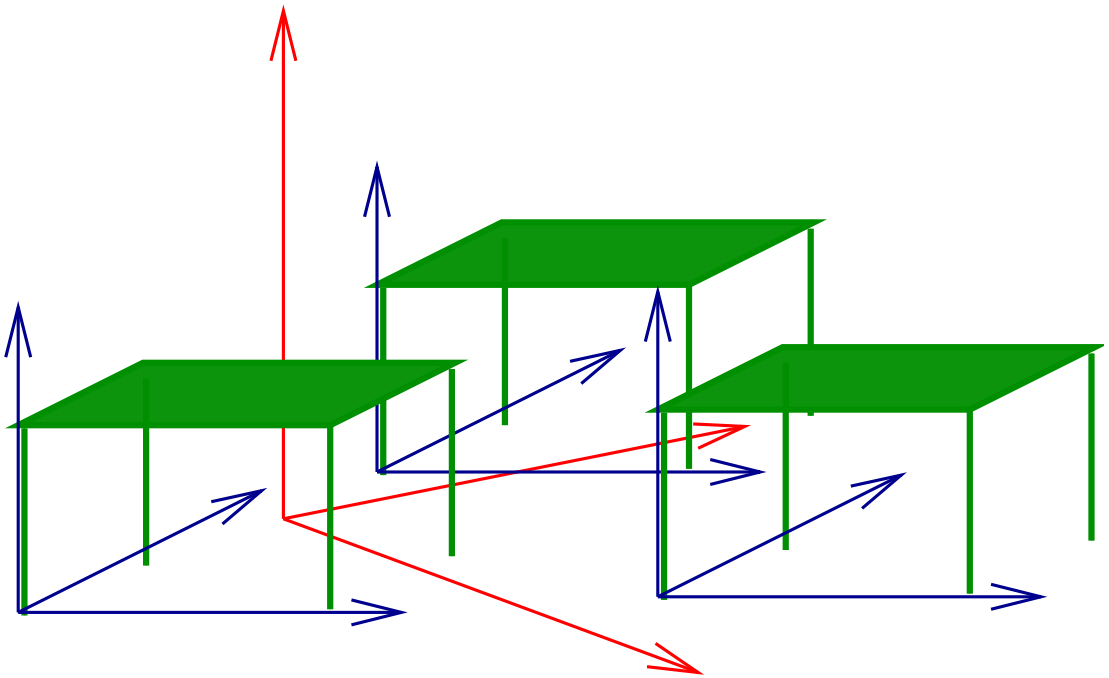
A Single Table

A table defined in its **own, natural** coordinate system.



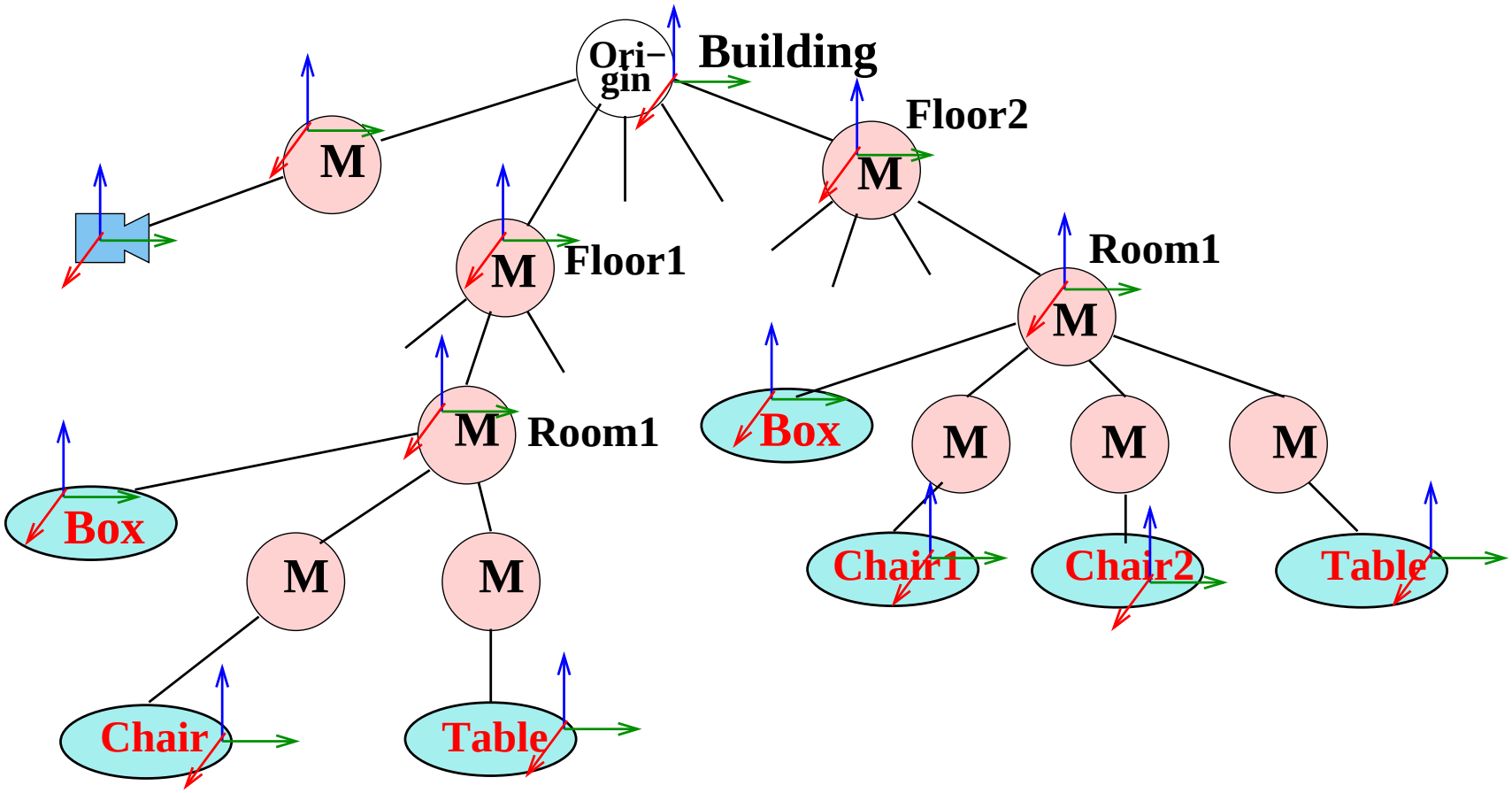
Many Tables in a Room

Place many tables in a **common world coord system**!



Start from world coord frame, move to each table's frame

A Hierarchical Building Model



Each matrix $\mathbf{M}$ aligns parent frame to child frame

Different Coordinates

- **Object Reference:** Object is described in an internal coordinate frame called **ORC**.
- **World:** Common reference frame to describe different objects, called **WC**.
- **Camera/View Reference:** Describe with respect to the current camera position/orientation, called **VRC**
- **How the scene appears to the camera** alone determines the image produced
- Role of Geometry in Computer Graphics:
  Describe the scene in the camera coordinate frame

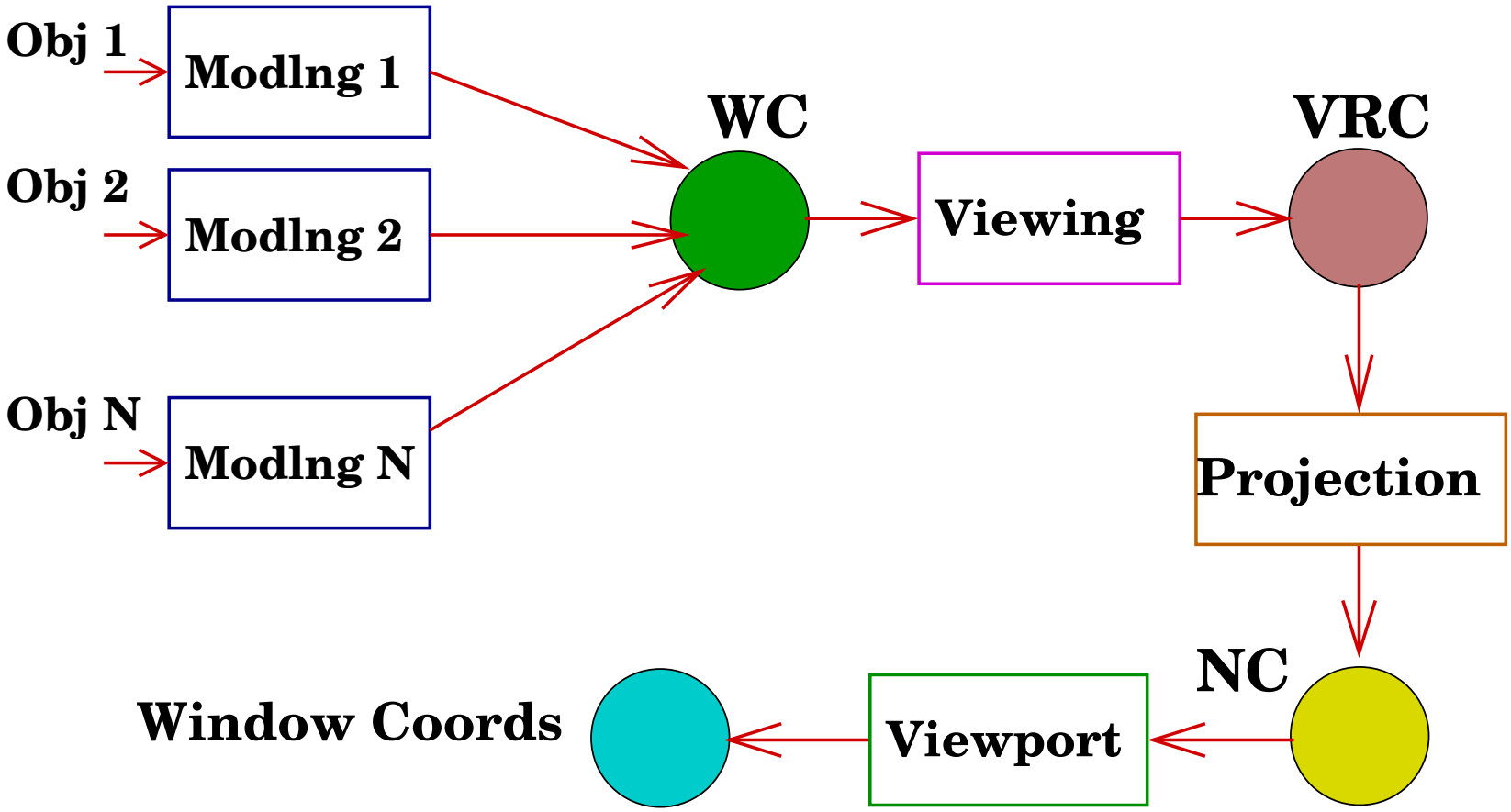
Transformations

- **Modelling:** Convert from object coordinates to world coordinates (ORC to WC).
- **View Orientation or Viewing:** From world coordinates to camera coordinates (WC to VRC).
- Simple coordinate transformations.
- **View Mapping or Projection:** From VRC to Normalized Coordinates (NC).
- **Viewport:** From NC to window coordinates.

3D Graphics Pipeline

- Objects are specified in their own coordinate system and placed in the world coordinate frame
  – Set up the world or the “set” as a movie director does
- Camera is also placed in the world coordinate frame.
  – That is where the camera person is!
- Image is what the camera sees of the world. Everything needs to come to the camera-based coordinate system
- Camera-to-world geometry is first projected to normalized coordinates and then to screen.

3D Graphics: Block Diagram



Different Coordinates

- **Object Reference:** Object is described in an internal coordinate frame called **ORC**.
- **World:** Common reference frame to describe different objects, called **WC**.
- **Camera/View Reference:** Describe with respect to the current camera position/orientation, called **VRC**.
- **Normalized Projection:** A standard space from which projection is easy, called **NPC**.
- **Screen:** Coordinates in the output device space.

Modelling and Viewing

- Transform points from object coordinates (ORC) to world coordinates (WC) to camera coordinates (VRC)
- A series of transformations for each object or point

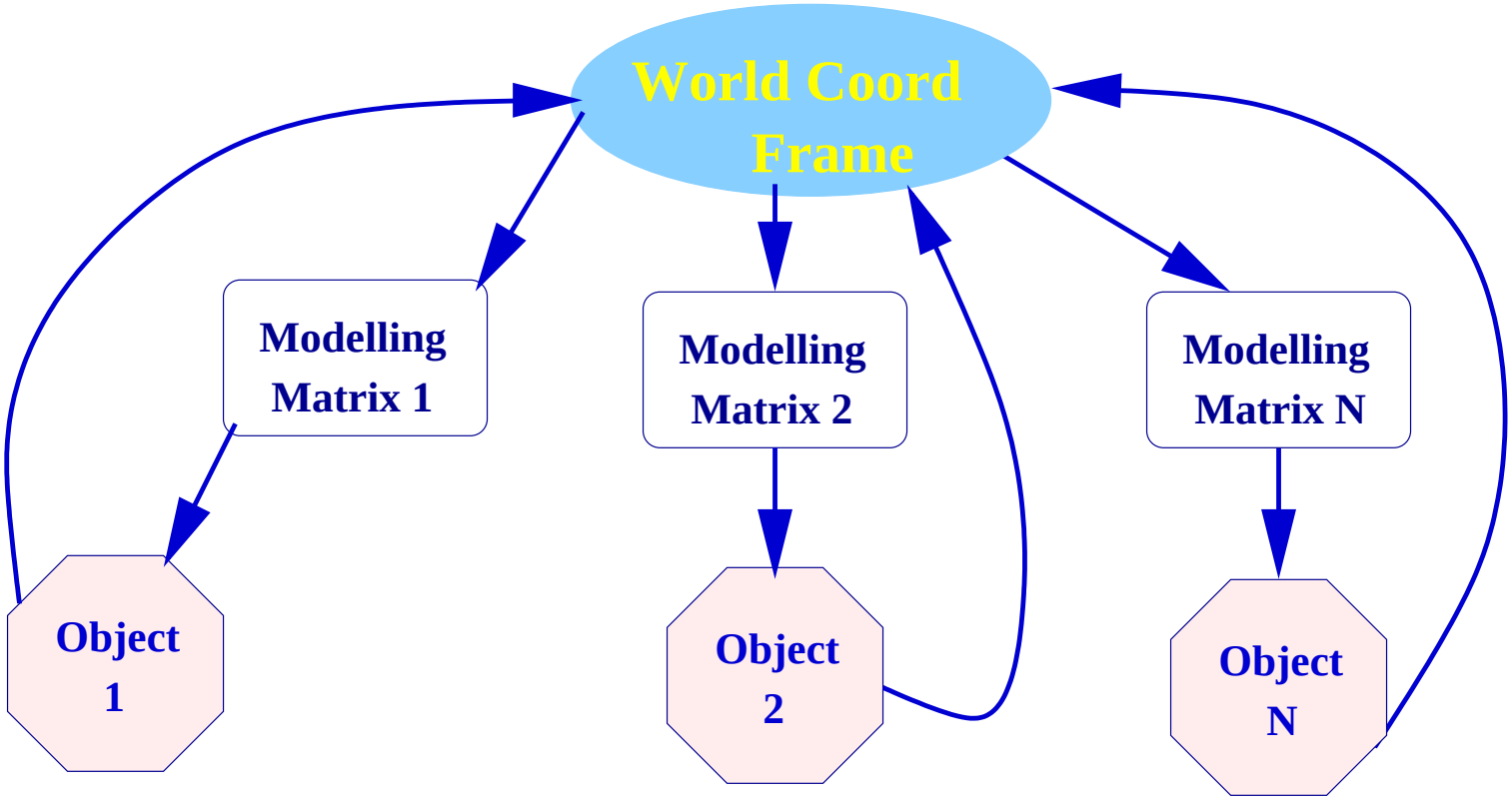
$$\mathbf{P_{VRC}} = \underset{VRC}{|\ \mathbf{V}} \underset{WC}{|\ \mathbf{M}} \underset{ORC}{|\ \mathbf{P_{ORC}}}$$

- Goal: Evaluate the coordinates of each point/line/triangle with respect to the camera

Modelling

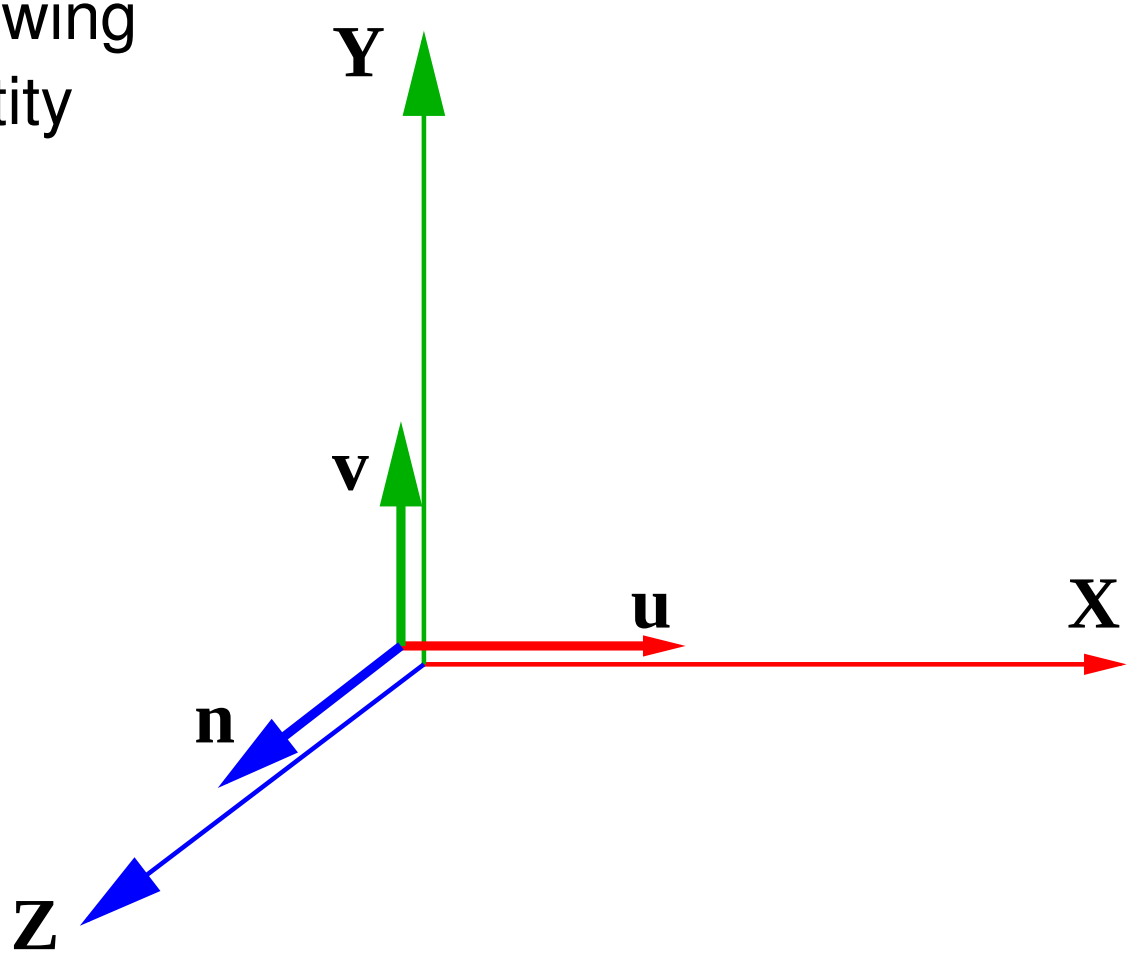
- Goal: Transform object coordinates to world coordinates.
- Method: Place ORC fame in the world coordinate frame.
- A single transformation matrix or **modelling matrix** with translation, rotation, scaling.
- A unit cube at origin can generate any cuboid using translation/rotation/scaling.
- Different objects have different modelling matrices.

Modelling Different Objects



When OpenGL starts

Modelling and Viewing
Matrices are Identity



Setting up Objects and Camera

- WC is at VRC at start. First push it away to where WC should be. This is the Viewing Transformation matrix $\mathbf{V}$

$$\mathbf{P_{VRC}} = \underset{VRC}{|\ \mathbf{V}} \underset{WC}{|\ \mathbf{M}} \underset{ORC}{|\ \mathbf{P_{ORC}}}$$

- **Stay here** and draw objects in the scene one by one
  – Move to ORC of each object and draw its own model
  – Each object i has its Modelling Matrix $\mathbf{M}_i$
- Create matrix $\mathbf{PVM}_i$ and send to shader
  – Draw the object using description in its own frame

Structure of an OpenGL Program

// Set projection matrix **P** (covered later)

// WC is aligned to VRC on start

// Camera is given by Pos & Orientation in WC

V = R(−Orient) T(−Pos); // WC moved away from VRC

// WC is set. Model each object with it as reference

// Draw object i with respect to WC

M = T(i) R(i) // Modelling matrix for object i

Mat = P V M // Form the MVP matrix

send Mat to shader // Send to shader

`drawObject(i)` // Draw object polygons

// Start next object with respect to WC

Geometry and Transformations

- Understand those to get things right!
- Two views possible, both useful:
  – coordinates change in a fixed reference frame
  – reference frames change, points get different coords
- Use both to understand how to set up different objects as well as the one camera in a common (world) coordinate frame